Java nâng cao và chuyên sâu

JVM, ClassLoader, Garbage Collection, Đa luồng, Concurrency và các chủ đề nâng cao khác

Tài liệu dành cho người đã vững Java Core (OOP, Collections, Exception, I/O cơ bản). Mục tiêu là hiểu cơ chế bên trong của ngôn ngữ Java và JVM, để viết mã đúng, đo đạc được hiệu năng và gỡ lỗi được những sự cố khó. Phiên bản tham chiếu: Java 17 và Java 21 (LTS).

Cấu trúc tài liệu

- Phần 1: Kiến trúc JVM, ClassLoader, vùng nhớ
- Phần 2: Garbage Collection và JIT
- Phần 3: Công cụ giám sát và tuning JVM
- Phần 4: Đa luồng cơ bản: vòng đời thread, synchronized, volatile, Java Memory Model
- Phần 5: java.util.concurrent, CompletableFuture, Virtual Threads
- Phần 6: Generics, Reflection, Annotation, Proxy, NIO, Stream nâng cao, Java hiện đại
- Phần 7: Lộ trình học và tài liệu tham khảo

PHẦN 1. KIẾN TRÚC JVM

1.1 JDK, JRE, JVM

- **JVM**: máy ảo thực thi bytecode, là đặc tả (JVM Specification) và có nhiều bản cài đặt (HotSpot, OpenJ9, GraalVM).
- **JRE**: JVM + thư viện chuẩn. Từ Java 11 Oracle không còn phát hành JRE riêng.
- **JDK**: JRE + công cụ phát triển (javac, jar, jlink, jcmd, jfr...).

Luồng từ mã nguồn đến chạy:

```
Foo.java --javac--> Foo.class (bytecode) --ClassLoader--> Runtime Data Areas
                                                              |
                                                Execution Engine (Interpreter + JIT + GC)
```

Bytecode độc lập nền tảng, còn JVM thì phụ thuộc nền tảng. Đó là cơ chế của Write Once, Run Anywhere. Xem bytecode bằng lệnh `javap -c -p Foo`.

1.2 ClassLoader

Một class đi qua 3 giai đoạn: **Loading → Linking → Initialization**.

Giai đoạn	Công việc
Loading	Đọc file .class, tạo đối tượng `Class` trong Metaspace
Linking: Verification	Kiểm tra bytecode hợp lệ, an toàn (cấu trúc, kiểu, stack)
Linking: Preparation	Cấp phát bộ nhớ cho static field, gán giá trị mặc định (0, null, false)
Linking: Resolution	Chuyển symbolic reference thành direct reference
Initialization	Chạy static initializer và gán giá trị static thật sự theo thứ tự trong mã

Class được khởi tạo (initialize) khi: tạo instance bằng new, gọi static method, truy cập static field không phải hằng số compile-time, gọi `Class.forName`, khởi tạo subclass (superclass khởi tạo trước), hoặc là class chứa `main`.

Lưu ý: hằng `static final` kiểu primitive/String gán bằng literal được inline lúc compile, nên truy cập nó **không** kích hoạt khởi tạo class.

Mô hình ủy quyền cha (Parent Delegation)

```
Bootstrap ClassLoader   (native, nạp java.base: java.lang.*, java.util.*...)
      ^
Platform ClassLoader    (Java 9+, thay Extension; nạp module nền tảng)
      ^
Application (System) ClassLoader  (nạp classpath của ứng dụng)
      ^
Custom ClassLoader      (do bạn viết)
```

Khi cần nạp class, loader hiện tại **hỏi loader cha trước**; chỉ khi cha không tìm thấy mới tự nạp. Lợi ích: tránh nạp trùng, và bảo vệ class lõi (không ai ghi đè được `java.lang.String`).

Một class được định danh bởi **tên đầy đủ + ClassLoader nạp nó**. Cùng tên nhưng khác loader là hai class khác nhau, đây là nguồn gốc của `ClassCastException` kỳ lạ trong container như Tomcat, OSGi.

```
public class MyLoader extends ClassLoader {
    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        byte[] data = loadBytes(name);          // tự đọc bytes từ file,
mạng, DB...
        return defineClass(name, data, 0, data.length);
    }
}
```

ClassNotFoundException vs NoClassDefFoundError

- `ClassNotFoundException` (checked Exception): nạp động qua `Class.forName`, `loadClass` mà không thấy class.
- `NoClassDefFoundError` (Error): class có lúc compile nhưng thiếu lúc chạy, hoặc khởi tạo static thất bại trước đó (`ExceptionInInitializerError` rồi lần sau là NoClassDefFoundError).

1.3 Runtime Data Areas (vùng nhớ của JVM)

Vùng	Phạm vi	Nội dung	Lỗi liên quan
Heap	Chung mọi thread	Object, mảng	OutOfMemoryError: Java heap space
Metaspace	Chung	Metadata của class, method, constant pool (Java 8+, nằm ở native memory, thay PermGen)	OutOfMemoryError: Metaspace
JVM Stack	Mỗi thread một stack	Các stack frame	StackOverflowError
PC Register	Mỗi thread	Địa chỉ lệnh bytecode đang chạy	
Native Method Stack	Mỗi thread	Phục vụ method native (JNI)	
Code Cache	Chung	Mã máy do JIT sinh ra	CodeCache is full

Vùng	Phạm vi	Nội dung	Lỗi liên quan
Direct Memory	Ngoài heap	ByteBuffer.allocateDirect, NIO	OutOfMemoryError: Direct buffer memory

Stack frame

Mỗi lần gọi method tạo một frame gồm: **Local Variable Array** (tham số, biến cục bộ, `this`), **Operand Stack** (nơi tính toán trung gian) và **Frame Data** (tham chiếu constant pool, thông tin xử lý exception). Biến primitive cục bộ nằm trên stack, còn đối tượng nằm trên heap và stack chỉ giữ tham chiếu.

Cấu trúc Heap (generational)

```
Heap
|-- Young Generation
|     |-- Eden        (object mới sinh)
|     |-- Survivor S0
|     |-- Survivor S1
|-- Old Generation (Tenured)   (object sống lâu)
```

Vòng đời object: sinh ở Eden → Minor GC → nếu còn sống chuyển sang Survivor và tăng tuổi (age) → đủ ngưỡng tuổi (`-XX:MaxTenuringThreshold` , tối đa 15) thì promote lên Old → Old đầy thì Major/Full GC. Object quá lớn có thể cấp phát thẳng vào Old.

String Pool

Chuỗi literal được lưu trong String Pool (năm trong heap từ Java 7). `intern()` đưa chuỗi vào pool.

```
String a = "java";
String b = "java";
String c = new String("java");
System.out.println(a == b);          // true  (cùng pool)
System.out.println(a == c);          // false (c năm ngoài pool)
System.out.println(a == c.intern()); // true
```

Escape Analysis

JIT phân tích nếu object không thoát khỏi method/thread thì có thể: **scalar replacement** (tách object thành các biến thường, không cấp phát heap), **lock elision** (bỏ synchronized thừa). Vì vậy câu nói mọi object đều năm trên heap chỉ đúng về mặt ngữ nghĩa, không đúng

tuyệt đối về mặt triển khai.

PHẦN 2. GARBAGE COLLECTION VÀ JIT

2.1 Nền tảng GC

Object còn sống là object có thể với tới được (reachable) từ **GC Roots**: biến cục bộ trên stack của thread, static field, JNI reference, các thread đang chạy, class đã nạp. Java **không dùng reference counting** (vì không xử lý được vòng tham chiếu) mà dùng truy vết tính khả đạt (tracing).

Ba thuật toán cơ bản:

- **Mark-Sweep**: đánh dấu sống, quét dọn chết. Nhanh nhưng gây phân mảnh.
- **Mark-Compact**: thêm bước dồn object sống lại. Hết phân mảnh nhưng tốn công di chuyển.
- **Copying**: chép object sống sang vùng trống, xóa cả vùng cũ. Nhanh, không phân mảnh nhưng tốn gấp đôi bộ nhớ. Dùng cho Young Gen (Eden + Survivor).

Stop-The-World (STW): mọi thread ứng dụng bị dừng tại *safepoint* để GC làm việc. Mục tiêu tuning là giảm thời gian dừng (latency) và tăng thông lượng (throughput).

Các loại GC event: Minor GC (Young), Major GC (Old), Mixed GC (G1: Young + một số vùng Old), Full GC (toàn heap, thường kèm Metaspace, đắt nhất).

2.2 Các bộ thu gom (Collector)

Collector	Cờ bật	Đặc điểm	Phù hợp
Serial	-XX:+UseSerialGC	1 thread, STW	Ứng dụng nhỏ, 1 CPU, container nhỏ
Parallel	-XX:+UseParallelGC	Nhiều thread GC, tối đa throughput	Batch, xử lý nền
G1	-XX:+UseG1GC (mặc định từ Java 9)	Chia heap thành region, đặt mục tiêu thời gian dừng	Đa dụng, heap vài GB đến hàng chục GB

Collector	Cờ bật	Đặc điểm	Phù hợp
ZGC	-XX:+UseZGC (Java 15 production; generational từ Java 21)	Concurrent gần như toàn bộ, dừng dưới 1ms	Heap rất lớn (đến TB), cần latency thấp
Shenandoah	-XX:+UseShenandoahGC	Compaction đồng thời, pause thấp	Latency thấp (OpenJDK)
Epsilon	-XX:+UseEpsilonGC	Không thu gom gì	Test hiệu năng, chương trình ngắn

G1 chi tiết: heap chia thành các region bằng nhau (1MB đến 32MB), mỗi region đóng vai trò Eden, Survivor, Old hoặc Humongous (object lớn hơn 50% kích thước region). G1 chọn các region có nhiều rác nhất để thu trước (Garbage First). Chu kỳ: Young-only → Concurrent Marking → Mixed collections. Mục tiêu: `-XX:MaxGCPauseMillis=200`.

2.3 Các loại tham chiếu (java.lang.ref)

Loại	Khi nào bị thu	Dùng cho
Strong	Không bao giờ khi còn reachable	Tham chiếu thông thường
SoftReference	Khi sắp thiếu bộ nhớ	Cache nhạy cảm bộ nhớ
WeakReference	Ở lần GC kế tiếp nếu chỉ còn weak	`WeakHashMap`, metadata gắn với object
PhantomReference	Sau khi object finalize, dùng với ReferenceQueue	Dọn tài nguyên thay finalize

`finalize()` đã **deprecated for removal** (Java 18+): không đảm bảo thời điểm chạy, làm chậm GC, có thể hồi sinh object. Thay bằng `try-with-resources` (AutoCloseable) hoặc `java.lang.ref.Cleaner`.

2.4 Memory leak trong Java

Java có GC nhưng vẫn rò rỉ khi object không còn dùng nhưng vẫn reachable:

- Collection `static` (Map, List) cứ thêm mà không xóa.
- Listener, callback đăng ký mà không hủy đăng ký.
- `ThreadLocal` không `remove()` trong thread pool (thread sống mãi, value sống mãi).

- Inner class không static giữ ngầm tham chiếu tới outer class.
- Key của HashMap mutable hoặc thiếu `equals/hashCode` đúng, gây phình map.
- Resource không đóng (Connection, Stream), ClassLoader leak khi redeploy.

Các dạng OutOfMemoryError thường gặp: `Java heap space`, `GC overhead limit exceeded`, `Metaspace`, `unable to create native thread`, `Direct buffer memory`.

2.5 JIT Compiler và Execution Engine

JVM chạy theo mô hình **hỗn hợp**: bắt đầu bằng **Interpreter** (khởi động nhanh, chạy chậm), phát hiện **hot spot** (method/loop chạy nhiều) rồi biên dịch sang mã máy.

- **Tiered Compilation** (mặc định): mức 0 interpreter; mức 1 đến 3 do **C1** (biên dịch nhanh, ít tối ưu, có profiling); mức 4 do **C2** (tối ưu mạnh, chậm biên dịch).
- **Tối ưu thường gặp**: method inlining, loop unrolling, dead code elimination, escape analysis, lock coarsening/elision, branch prediction theo profile.
- **Deoptimization**: nếu giả định lúc tối ưu sai (ví dụ class mới nạp làm phá giả định monomorphic call), JVM quay về interpreter.
- **Warm-up**: ứng dụng chạy chậm lúc đầu rồi nhanh dần, nên benchmark phải dùng **JMH**, không dùng `System.currentTimeMillis` thô.
- Các hướng giảm thời gian khởi động: AppCDS (Class Data Sharing), GraalVM Native Image (AOT), CRaC.

PHẦN 3. CÔNG CỤ GIÁM SÁT VÀ TUNING JVM

3.1 Cờ JVM quan trọng

```
-Xms2g -Xmx2g                  # heap ban đầu = tối đa, tránh resize
-Xss512k                       # kích thước stack mỗi thread
-XX:MaxMetaspaceSize=256m
-XX:+UseG1GC -XX:MaxGCPauseMillis=200
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/dump.hprof
-Xlog:gc*:file=gc.log:time,uptime   # GC log thống nhất (Java 9+)
-XX:+PrintFlagsFinal           # xem toàn bộ cờ và giá trị hiện hành
-XX:MaxRAMPercentage=75        # tính heap theo giới hạn RAM của container
```

3.2 Công cụ dòng lệnh trong JDK

Công cụ	Công dụng
jps	Liệt kê tiến trình Java
jstat -gcutil PID 1000	Thống kê GC, tỷ lệ dùng từng vùng mỗi giây
jstack PID	Thread dump, tìm deadlock, thread bị kẹt
jmap -dump:live,format=b,file=x.hprof PID	Heap dump
jcmd PID GC.heap_info / Thread.print / VM.flags	Công cụ đa năng thay thế dần các lệnh cũ
jfr / Java Flight Recorder	Ghi profile chi phí thấp chạy ngay trên production
javap	Xem bytecode
jlink / jpackage	Tạo runtime tối giản và bộ cài

GUI: **VisualVM**, **JDK Mission Control** (đọc file JFR), **Eclipse MAT** (phân tích heap dump, tìm leak qua Dominator Tree và Leak Suspects), **JMH** (microbenchmark).

3.3 Quy trình xử lý sự cố điển hình

1. **CPU cao**: `top -H -p PID` tìm thread tốn CPU → đổi thread id sang hex → khớp với `nid` trong `jstack`.
2. **OOM / heap tăng mãi**: bật HeapDumpOnOutOfMemoryError → mở bằng MAT → xem Dominator Tree.
3. **Ứng dụng đơ**: chụp 3 thread dump cách nhau vài giây, tìm thread BLOCKED, WAITING và deadlock.
4. **Pause dài**: bật GC log, xem nguyên nhân (Full GC? Humongous allocation? to-space exhausted?), rồi điều chỉnh heap, region, hoặc đổi collector.

Nguyên tắc tuning: đo trước khi chỉnh, mỗi lần chỉ đổi một tham số, ưu tiên sửa mã (giảm cấp phát, tránh object tạm) trước khi chỉnh cờ.

PHẦN 4. ĐA LUỒNG (MULTITHREADING) CHUYÊN SÂU

4.1 Vòng đời của Thread

```
NEW --start()--> RUNNABLE <--> BLOCKED     (chờ giành monitor lock)
                    |  ^   <--> WAITING     (wait(), join(), park())
                    |  |   <--> TIMED_WAITING (sleep(n), wait(n), join(n))
                    v  |
                 TERMINATED
```

- `RUNNABLE` gồm cả trạng thái đang chạy lẫn sẵn sàng chạy (Java không tách hai trạng thái này).
- Gọi `start()` hai lần ném `IllegalThreadStateException`. Gọi `run()` trực tiếp chỉ là gọi method thường, không tạo thread mới.
- `sleep()` giữ nguyên lock; `wait()` nhả lock của monitor đang giữ.
- `interrupt()` chỉ đặt cờ ngắt. Thread đang `sleep/wait/join` sẽ ném `InterruptedException` và xóa cờ. Quy tắc vàng: **không nuốt InterruptedException**, hoặc ném tiếp, hoặc gọi lại `Thread.currentThread().interrupt()`.
- Daemon thread (`setDaemon(true)`) không giữ JVM sống; JVM thoát khi hết thread non-daemon.

4.2 Race condition và tính nguyên tử

`count++` gồm 3 bước (đọc, cộng, ghi) nên không nguyên tử. Hai thread có thể cùng đọc một giá trị và làm mất một lần cập nhật.

Ba vấn đề cốt lõi của đa luồng:

1. **Atomicity** (tính nguyên tử): thao tác không bị chen ngang.
2. **Visibility** (tính hiển thị): thread này ghi thì thread kia thấy được.
3. **Ordering** (thứ tự): trình biên dịch, JIT và CPU có thể sắp xếp lại lệnh miễn kết quả trong một thread không đổi.

4.3 synchronized

```
public synchronized void a() { }                // khóa trên this
public static synchronized void b() { }         // khóa trên Foo.class
public void c() { synchronized (lockObj) { } }  // khóa trên object chỉ định
```

- Mỗi object có một **monitor**. `synchronized` đảm bảo **mutual exclusion** (loại trừ lẫn nhau) và **visibility** (thoát khối synchronized thì mọi thay đổi được đẩy ra bộ nhớ chính, vào khối thì đọc lại giá trị mới).
- **Reentrant**: cùng một thread có thể vào lại khối đã khóa.
- Không thể ngắt khi đang chờ lock, không có timeout, không công bằng (khác `ReentrantLock`).
- JVM tối ưu lock theo mức: **biased lock** (đã bị loại bỏ từ Java 18) → **lightweight lock** (CAS) → **heavyweight lock** (mutex hệ điều hành). Lock chỉ "phình" lên khi có tranh chấp.
- Không dùng String literal, Integer cache hay object dùng chung làm khóa.

4.4 wait / notify

Chỉ gọi được khi đang giữ monitor, nếu không sẽ ném `IllegalMonitorStateException`. **Luôn đặt wait trong vòng lặp while** để chống *spurious wakeup*.

```
synchronized (lock) {
    while (queue.isEmpty()) {
        lock.wait();            // nhả lock và chờ
    }
    item = queue.poll();
}
// bên producer
synchronized (lock) {
    queue.add(x);
    lock.notifyAll();           // ưu tiên notifyAll hơn notify
}
```

4.5 volatile

- Đảm bảo **visibility** và **chống sắp xếp lại** quanh biến đó (có memory barrier).
- **Không** đảm bảo atomicity cho `count++`.
- Dùng đúng cho: cờ trạng thái (`volatile boolean running`), publish một object bất biến, và trong Double-Checked Locking.

```
public class Singleton {
    private static volatile Singleton instance;   // volatile là bắt buộc
    public static Singleton get() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Vì sao cần volatile: `new Singleton()` gồm cấp phát, gọi constructor, gán tham chiếu. Nếu bị sắp xếp lại, thread khác có thể thấy `instance != null` nhưng object chưa được khởi tạo xong. (Cách gọn hơn: enum singleton hoặc holder class.)

4.6 Java Memory Model (JMM) và happens-before

JMM định nghĩa khi nào ghi của thread này **chắc chắn** thấy được ở thread khác. Khái niệm trung tâm là quan hệ **happens-before**: nếu A happens-before B thì kết quả của A hiển thị với B.

Các quy tắc happens-before chính:

- **Program order**: trong một thread, lệnh trước happens-before lệnh sau.
- **Monitor lock**: unlock một monitor happens-before lock tiếp theo trên cùng monitor.
- **Volatile**: ghi biến volatile happens-before đọc biến đó sau đó.
- **Thread start**: `t.start()` happens-before mọi hành động trong t.
- **Thread join**: mọi hành động trong t happens-before khi `t.join()` trả về.
- **Transitivity**: A hb B và B hb C thì A hb C.
- **final field**: gán xong final field trong constructor thì thread nào thấy tham chiếu object (không để `this` bị rò rỉ) cũng thấy giá trị đó.

Không có quan hệ happens-before thì chương trình có **data race** và hành vi không xác định.

4.7 Deadlock, livelock, starvation

Deadlock xảy ra khi đủ 4 điều kiện Coffman: mutual exclusion, hold and wait, no preemption, circular wait. Cách phòng tránh phổ biến nhất: **luôn lấy lock theo thứ tự cố định**; hoặc dùng `tryLock(timeout)`; giảm phạm vi lock; tránh gọi code lạ (callback) khi đang giữ lock.

```
// Thread 1: lock A rồi B        Thread 2: lock B rồi A   --> deadlock
```

Phát hiện: `jstack PID` sẽ in rõ dòng "Found one Java-level deadlock".

- **Livelock**: các thread vẫn chạy nhưng liên tục nhường nhau, không tiến triển.
- **Starvation**: thread không bao giờ giành được tài nguyên (ví dụ lock không công bằng, ưu tiên thấp).

4.8 ThreadLocal

Mỗi thread có bản sao riêng của biến. Dùng cho context theo request (user, transaction), `SimpleDateFormat` (không thread-safe). **Trong thread pool phải gọi `remove()`** trong khối finally để tránh rò rỉ và lẫn dữ liệu giữa các request. Với virtual threads nên cân nhắc `ScopedValue` (preview/final tùy phiên bản).

4.9 Thread-safety và bất biến

Cách đạt thread-safe, xếp theo độ ưu tiên: (1) **bất biến** (immutable: `final`, không setter, copy phòng thủ, ví dụ `String`, `record`); (2) **không chia sẻ trạng thái** (confinement, ThreadLocal, biến cục bộ); (3) dùng lớp có sẵn (`Concurrent*`, `Atomic*`); (4) đồng bộ hóa thủ công (`synchronized`, `Lock`).

PHẦN 5. JAVA.UTIL.CONCURRENT

5.1 Executor Framework và ThreadPool

Không nên tự `new Thread()` cho từng tác vụ: tạo thread tốn kém và không kiểm soát được số lượng. Dùng pool.

```
ExecutorService pool = Executors.newFixedThreadPool(8);
Future<Integer> f = pool.submit(() -> compute());
int result = f.get(5, TimeUnit.SECONDS);   // chặn, có timeout
pool.shutdown();                           // ngừng nhận việc mới, chạy nốt
việc cũ
pool.awaitTermination(30, TimeUnit.SECONDS);
```

Factory	Đặc điểm	Rủi ro
newFixedThreadPool(n)	n thread, hàng đợi **không giới hạn**	Hàng đợi phình gây OOM
newCachedThreadPool()	Số thread không giới hạn, thread rảnh 60s thì hủy	Quá nhiều thread khi tải cao
newSingleThreadExecutor()	1 thread, tuần tự	Hàng đợi không giới hạn
newScheduledThreadPool(n)	Chạy trễ hoặc định kỳ	
newWorkStealingPool()	Dựa trên ForkJoinPool	
newVirtualThreadPerTaskExecutor()	Mỗi task một virtual thread (Java 21)	

Trong sản xuất nên tự dựng `ThreadPoolExecutor` để kiểm soát:

```
new ThreadPoolExecutor(
    corePoolSize, maximumPoolSize,
    keepAliveTime, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(1000),               // hàng đợi CÓ GIỚI HẠN
    namedThreadFactory,                           // đặt tên thread để dễ
debug
    new ThreadPoolExecutor.CallerRunsPolicy());   // chính sách từ chối
```

Thứ tự xử lý khi có task mới: còn thread core thì tạo thread core → core đầy thì **xếp vào hàng đợi** → hàng đợi đầy thì tạo thêm thread đến `maximumPoolSize` → vượt quá thì áp **RejectedExecutionHandler** (Abort, CallerRuns, Discard, DiscardOldest).

Kích thước pool gợi ý: tác vụ tính toán nặng CPU ≈ số core (+1); tác vụ nhiều chờ I/O ≈ core × (1 + thời gian chờ / thời gian tính).

`Runnable` không trả kết quả và không ném checked exception; `Callable<V>` có trả kết quả và có thể ném exception; `Future` đại diện kết quả tương lai (`get`, `cancel`, `isDone`).

5.2 Lock API

```
ReentrantLock lock = new ReentrantLock(true);   // true = công bằng (fair)
lock.lock();
try {
    // vùng găng
} finally {
    lock.unlock();                              // luôn unlock trong finally
}
```

Ưu điểm so với `synchronized`: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, lock công bằng, và nhiều `Condition` trên một lock (tương đương nhiều wait-set).

- **ReadWriteLock** (`ReentrantReadWriteLock`): nhiều reader song song, writer độc quyền. Hợp với dữ liệu đọc nhiều ghi ít.
- **StampedLock**: hỗ trợ **optimistic read** (đọc không khóa rồi kiểm tra `validate(stamp)`), nhanh hơn nhưng không reentrant và phức tạp hơn.
- **LockSupport.park/unpark**: nền tảng cấp thấp mà các lock khác xây lên.

5.3 Atomic và CAS

CAS (Compare-And-Swap) là lệnh CPU nguyên tử: chỉ ghi giá trị mới nếu giá trị hiện tại bằng giá trị kỳ vọng. Các lớp `AtomicInteger`, `AtomicLong`, `AtomicReference` xây trên CAS (qua `VarHandle` / `Unsafe`) nên **lock-free**.

```
AtomicInteger n = new AtomicInteger();
n.incrementAndGet();
n.updateAndGet(x -> x * 2);
n.compareAndSet(5, 10);
```

- **Vấn đề ABA**: giá trị đổi A→B→A, CAS tưởng không đổi. Khắc phục bằng `AtomicStampedReference`.
- Khi tranh chấp rất cao, dùng **`LongAdder`** (chia nhỏ thành nhiều ô cộng dồn) nhanh hơn `AtomicLong`.

5.4 Synchronizer

Lớp	Ý nghĩa	Tái sử dụng
CountDownLatch	Chờ N sự kiện hoàn thành rồi mới đi tiếp	Không (dùng 1 lần)
CyclicBarrier	N thread cùng chờ nhau tại điểm hẹn	Có
Semaphore	Giới hạn số thread vào đồng thời (cấp phép)	Có
Phaser	Barrier nhiều pha, số bên tham gia thay đổi động	Có
Exchanger	Hai thread hoán đổi dữ liệu	Có

5.5 Concurrent Collections

Lớp	Ghi chú
ConcurrentHashMap	Từ Java 8 khóa theo từng bucket (CAS + synchronized trên node đầu), đọc không khóa. Không cho key hoặc value null. Dùng `compute`, `merge`, `putIfAbsent` thay vì check-then-put
CopyOnWriteArrayList	Mỗi lần ghi sao chép mảng; đọc cực nhanh, hợp ít ghi (listener)
ConcurrentLinkedQueue / Deque	Hàng đợi lock-free
BlockingQueue: ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue, SynchronousQueue, DelayQueue	`put/take` chặn; nền tảng của mô hình producer-consumer
ConcurrentSkipListMap	Map có thứ tự, thread-safe

`Collections.synchronizedList/Map` chỉ đồng bộ từng method, duyệt vẫn phải tự khóa. `HashMap` dùng đa luồng có thể mất dữ liệu (Java 7 còn gây vòng lặp vô hạn khi resize).

5.6 Fork/Join

Chia để trị: `RecursiveTask<V>` (có kết quả) và `RecursiveAction`. Pool dùng **work-stealing**: thread rảnh "đánh cắp" task từ cuối deque của thread bận. `parallelStream()` và `CompletableFuture` mặc định chạy trên `ForkJoinPool.commonPool()`.

```
class SumTask extends RecursiveTask<Long> {
    final int[] a; final int lo, hi;
    SumTask(int[] a, int lo, int hi) { this.a = a; this.lo = lo; this.hi =
hi; }
    protected Long compute() {
        if (hi - lo <= 10_000) { long s = 0; for (int i = lo; i < hi; i++) s
+= a[i]; return s; }
        int mid = (lo + hi) >>> 1;
        SumTask left = new SumTask(a, lo, mid), right = new SumTask(a, mid,
hi);
        left.fork();                        // chạy bất đồng bộ
        return right.compute() + left.join();
    }
}
```

5.7 CompletableFuture

Lập trình bất đồng bộ không chặn, nối chuỗi và kết hợp tác vụ.

```
CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(() -> fetchUser(id), executor)      // chạy async
    .thenApply(user -> user.getName())               // biến đổi (đồng bộ)
    .thenCompose(name -> fetchOrdersAsync(name))     // nối tiếp một CF khác
(flatMap)
    .exceptionally(ex -> "mặc định");                // xử lý lỗi

CompletableFuture.allOf(cf1, cf2).join();            // chờ tất cả
CompletableFuture.anyOf(cf1, cf2);                   // lấy cái xong đầu tiên
cf1.thenCombine(cf2, (a, b) -> a + b);               // gộp hai kết quả độc
lập
```

Phân biệt: `thenApply` (hàm thường), `thenCompose` (hàm trả về CompletableFuture, tránh lồng), `thenAccept` (tiêu thụ, không trả), `handle` (xử lý cả kết quả lẫn lỗi), `whenComplete` (quan sát, không đổi kết quả). Hậu tố `Async` chuyển bước đó sang pool khác. **Luôn truyền executor riêng** cho I/O để không làm cạn commonPool.

5.8 Virtual Threads (Java 21, Project Loom)

Virtual thread là thread nhẹ do **JVM lập lịch** (mount lên một số ít *carrier thread* của ForkJoinPool), tạo được hàng triệu cái. Khi gặp I/O chặn, nó được *unmount* để carrier chạy việc khác.

```
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        exec.submit(() -> { Thread.sleep(Duration.ofSeconds(1)); return 1;
});
    }
}   // đóng executor sẽ chờ mọi task xong

Thread.ofVirtual().start(() -> doWork());
```

- Phù hợp: tác vụ **chủ yếu chờ I/O** (HTTP, DB). Không giúp gì cho tác vụ nặng CPU.
- **Không pool** virtual thread; tạo mới cho mỗi tác vụ. Dùng `Semaphore` nếu cần giới hạn đồng thời.
- Cạm bẫy **pinning**: virtual thread bị "ghim" vào carrier khi chặn bên trong khối `synchronized` hoặc gọi native (đã được cải thiện ở các bản mới). Với lock nên ưu tiên `ReentrantLock`.
- Cho phép viết code đồng bộ dễ đọc mà vẫn đạt khả năng mở rộng như bất đồng bộ.
- **Structured Concurrency** (`StructuredTaskScope`, preview ở Java 21): coi nhóm task con như một đơn vị, hủy và xử lý lỗi đồng bộ với task cha.

PHẦN 6. CÁC CHỦ ĐỀ NÂNG CAO KHÁC

6.1 Generics chuyên sâu

Type erasure: thông tin kiểu generic bị xóa lúc compile, JVM chỉ thấy kiểu thô (raw) hoặc cận trên. Hệ quả:

- Không `new T()`, không `new T[10]`, không `instanceof List<String>`.
- `List<String>` và `List<Integer>` cùng là `List` lúc chạy.
- Hai method chỉ khác tham số generic sẽ bị trùng chữ ký sau khi xóa kiểu.
- Compiler sinh **bridge method** để giữ tính đa hình sau erasure.

Wildcard và quy tắc PECS (Producer Extends, Consumer Super)

```
List<? extends Number> src;   // chỉ ĐỌC (producer): lấy ra là Number, không
add được (trừ null)
List<? super Integer>  dst;   // chỉ GHI (consumer): add được Integer, đọc ra
chỉ là Object

static <T> void copy(List<? super T> dst, List<? extends T> src) {
    for (T t : src) dst.add(t);
}
```

Mảng có tính **covariant** (`Integer[]` là `Number[]` , có thể gây `ArrayStoreException` lúc chạy), còn generics **invariant** (`List<Integer>` không phải `List<Number>`) nên an toàn kiểu ngay lúc compile. Có thể giới hạn nhiều cận: `<T extends Number & Comparable<T>>` .

6.2 Reflection

Truy xuất và thao tác class, field, method, constructor ngay lúc chạy. Là nền tảng của Spring, Hibernate, JUnit, Jackson.

```
Class<?> c = Class.forName("com.demo.User");
Object o = c.getDeclaredConstructor().newInstance();
Method m = c.getDeclaredMethod("setName", String.class);
m.invoke(o, "An");
Field f = c.getDeclaredField("secret");
f.setAccessible(true);            // vượt private (bị hạn chế bởi module
system từ Java 9+)
```

Nhược điểm: chậm hơn gọi trực tiếp, mất kiểm tra lúc compile, phá vỡ đóng gói. Thay thế hiện đại, nhanh hơn: **MethodHandle** và **VarHandle** (`java.lang.invoke`).

6.3 Annotation

```
@Retention(RetentionPolicy.RUNTIME)     // SOURCE, CLASS, RUNTIME
@Target({ElementType.METHOD, ElementType.TYPE})
@Documented
@Inherited
public @interface Audit {
    String value() default "";
    int level() default 1;
}
```

Đọc lúc chạy qua reflection: `method.getAnnotation(Audit.class)`. Chỉ `RetentionPolicy.RUNTIME` mới đọc được lúc chạy. Annotation processing lúc compile (`javax.annotation.processing`) là cơ chế của Lombok, MapStruct.

6.4 Dynamic Proxy và nền tảng của AOP

JDK Dynamic Proxy (chỉ cho **interface**):

```
UserService proxy = (UserService) Proxy.newProxyInstance(
    loader, new Class<?>[]{ UserService.class },
    (p, method, args) -> {
        System.out.println("trước: " + method.getName());
        Object r = method.invoke(target, args);
        System.out.println("sau");
        return r;
    });
```

CGLIB / ByteBuddy: sinh subclass lúc chạy nên proxy được cả class không có interface (không proxy được class `final` hoặc method `final`). Spring AOP dùng hai cách này. Đây cũng là lý do gọi `this.method()` nội bộ trong cùng bean làm **mất** `@Transactional` hay `@Async`.

6.5 Java I/O nâng cao: NIO và NIO.2

Thành phần	Ý nghĩa
Buffer (ByteBuffer...)	Vùng dữ liệu có `position`, `limit`, `capacity`; `flip()` để chuyển từ ghi sang đọc. Có heap buffer và direct buffer
Channel	Kênh hai chiều (FileChannel, SocketChannel), đọc ghi qua Buffer
Selector	Một thread quản lý nhiều channel (I/O multiplexing, dựa trên epoll/kqueue). Nền tảng của Netty
Files, Path (NIO.2)	API file hiện đại: `Files.readAllLines`, `Files.walk`, `Files.copy`, `WatchService`
AsynchronousChannel	I/O bất đồng bộ (callback hoặc Future)

Phân biệt: **blocking I/O** (thread chờ), **non-blocking I/O** (trả về ngay, hỏi lại sau), **asynchronous I/O** (hệ điều hành báo khi xong). `FileChannel.transferTo` dùng **zero-copy** để chép file nhanh, không đi qua bộ nhớ user space.

6.6 Serialization

- Implement `Serializable`; `serialVersionUID` để kiểm tra tương thích phiên bản class.
- `transient` và `static` field không được tuần tự hóa.
- Constructor của class **không** Serializable ở phía cha vẫn chạy khi giải tuần tự hóa; constructor của class Serializable thì **không** chạy.
- Tùy biến bằng `writeObject/readObject`, `readResolve` (bảo vệ Singleton), hoặc `Externalizable`.
- **Bảo mật**: giải tuần tự hóa dữ liệu không tin cậy là lỗ hổng nghiêm trọng (RCE). Dùng `ObjectInputFilter`; ưu tiên JSON/Protobuf thay cho Java serialization.

6.7 Lambda, Functional Interface và Stream nâng cao

Lambda biên dịch bằng `invokedynamic` (không sinh inner class), nên nhẹ và JVM tối ưu được. Lambda chỉ capture biến cục bộ **effectively final**.

Các functional interface cốt lõi: `Function<T,R>`, `Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `UnaryOperator<T>`, `BiFunction`, kèm phiên bản primitive (`IntFunction`, `ToLongFunction`...) để tránh boxing.

```
Map<String, Double> avgByDept = employees.stream()
    .filter(e -> e.getSalary() > 0)
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));

List<String> flat =
lists.stream().flatMap(List::stream).distinct().sorted().toList();

String joined = names.stream().collect(Collectors.joining(", ", "[", "]"));
var parts = nums.stream().collect(Collectors.partitioningBy(n -> n % 2 ==
0));
```

Điểm cần nắm:

- Stream **lazy**: thao tác trung gian (`filter` , `map` , `sorted`) chỉ chạy khi có thao tác kết thúc (`collect` , `forEach` , `reduce`). Stream chỉ dùng được **một lần**.
- Thao tác **stateless** (`map` , `filter`) khác **stateful** (`sorted` , `distinct`).
- **Short-circuit**: `findFirst` , `anyMatch` , `limit` .
- `map` vs `flatMap` ; `reduce` với identity, accumulator, combiner (cho parallel).
- **parallelStream** chỉ đáng dùng khi dữ liệu lớn, thao tác nặng CPU, không có side effect và nguồn dễ chia nhỏ (ArrayList tốt, LinkedList kém). Mặc định dùng commonPool, dễ gây tranh chấp.
- `Optional` : dùng làm kiểu trả về, tránh dùng cho field hoặc tham số; tránh `get()` trần, ưu tiên `orElse` , `orElseGet` , `orElseThrow` , `map` .

6.8 Java hiện đại (Java 9 đến 21)

Tính năng	Phiên bản	Ý nghĩa
Module System (JPMS, `module-info.java`)	9	Đóng gói mạnh: `requires` , `exports` , `opens`
`var` (suy luận kiểu biến cục bộ)	10	Ít code thừa, vẫn là kiểu tĩnh
HTTP Client chuẩn	11	Hỗ trợ HTTP/2, bất đồng bộ
Switch expression	14	`case X -> value` , có `yield` , không fall-through
Text block	15	Chuỗi nhiều dòng bằng `"""`
Record	16	Lớp dữ liệu bất biến, tự sinh constructor, accessor, equals, hashCode, toString
Pattern matching `instanceof`	16	`if (o instanceof String s)`
Sealed classes	17	`sealed ... permits` : kiểm soát tập lớp con
Pattern matching cho switch, record pattern	21	Switch theo kiểu, giải cấu trúc record
Virtual Threads	21	Xem mục 5.8
Sequenced Collections	21	`getFirst()` , `getLast()` , `reversed()`

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

double area(Shape s) {
    return switch (s) {                      // đầy đủ, không cần default
nhờ sealed
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
    };
}
```

6.9 Một số cạm bẫy kinh điển

- `Integer` cache từ -128 đến 127: `Integer a = 127, b = 127; a == b` là true, nhưng với 128 là false. Luôn so sánh bằng `equals`.
- Ghi đè `equals` phải ghi đè `hashCode` (hợp đồng: hai object equals thì cùng hashCode). Key của HashMap nên bất biến.
- `HashMap`: bảng băm + danh sách liên kết, chuyển sang cây đỏ đen khi bucket có từ 8 phần tử và bảng từ 64 bucket; load factor 0.75; kích thước bảng luôn là lũy thừa của 2.
- `static` và `instance initializer`, thứ tự khởi tạo: static của cha → static của con → instance block và constructor của cha → instance block và constructor của con.
- `finally` luôn chạy (trừ `System.exit` hoặc JVM chết); `return` trong finally nuốt exception.
- Số thực: `0.1 + 0.2 != 0.3`, dùng `BigDecimal(String)` cho tiền tệ.
- `String` bất biến; nối chuỗi trong vòng lặp dùng `StringBuilder` (`StringBuffer` đồng bộ nhưng chậm).
- `ConcurrentModificationException`: sửa collection khi đang duyệt (cơ chế fail-fast qua `modCount`); dùng `Iterator.remove()` hoặc `removeIf`.

PHẦN 7. LỘ TRÌNH HỌC VÀ TÀI LIỆU THAM KHẢO

7.1 Lộ trình học gợi ý

1. **Tuần 1 đến 2**: JVM (ClassLoader, vùng nhớ, GC). Thực hành: chạy chương trình gây OOM, mở heap dump bằng MAT, đọc GC log.

2. **Tuần 3 đến 4**: Thread, synchronized, volatile, JMM. Thực hành: tự viết producer-consumer bằng wait/notify, tái hiện deadlock rồi dùng jstack để bắt.
3. **Tuần 5 đến 6**: java.util.concurrent, ThreadPoolExecutor, CompletableFuture, ConcurrentHashMap. Thực hành: benchmark bằng JMH.
4. **Tuần 7**: Generics, Reflection, Annotation, Proxy: tự viết một mini-framework DI hoặc AOP.
5. **Tuần 8**: Java hiện đại (record, sealed, pattern matching, virtual threads) và tổng hợp toàn bộ bằng một dự án nhỏ.

7.2 Tài liệu tham khảo nên đọc

- *Effective Java* (Joshua Bloch): chuẩn mực viết Java đúng cách.
- *Java Concurrency in Practice* (Brian Goetz): sách kinh điển về đa luồng.
- *Java Performance* (Scott Oaks) và *Optimizing Java*: tuning JVM và GC.
- *The Java Virtual Machine Specification* và *The Java Language Specification* trên docs.oracle.com.
- Các JEP tại openjdk.org/jeps (G1, ZGC, Virtual Threads, Pattern Matching...).

Gợi ý: học theo kiểu đọc một phần, viết code thử ngay, rồi dùng công cụ (jstack, jcmd, MAT, JFR) để quan sát tận mắt. Kiến thức JVM và concurrency chỉ thực sự vững khi bạn đã tự tái hiện và đo đạc được hiện tượng.

PHẦN MỞ RỘNG: ĐI SÂU VÀO CƠ CHẾ BÊN TRONG

Các Phần 1 đến 7 ở trên là bản khung bao quát. Các phần 8 và 9 dưới đây giải thích **cơ chế bên trong** (internals) của những chủ đề quan trọng nhất của Java và JVM, cần cho việc viết mã đúng và xử lý sự cố thực tế.

PHẦN 8. ĐI SÂU VÀO JVM

8.1 Cấu trúc file .class và bytecode

Một file .class gồm theo thứ tự:

Thành phần	Nội dung
Magic number	`0xCAFEBABE`
Version	minor và major (Java 8 = 52, 11 = 55, 17 = 61, 21 = 65). JVM cũ không chạy được class có version mới hơn: lỗi `UnsupportedClassVersionError`
Constant pool	Bảng hằng: tên class, tên method, chuỗi, số... mọi chỗ khác tham chiếu bằng chỉ số vào bảng này
Access flags	public, final, interface, abstract...
this_class, super_class, interfaces	Quan hệ kế thừa
Fields, Methods	Mô tả field và method (kèm thuộc tính `Code`)
Attributes	SourceFile, LineNumberTable, InnerClasses, Signature (generic), RuntimeVisibleAnnotations...

Ví dụ method đơn giản:

```
int add(int a, int b) {
    int c = a + b;
    return c;
}
```

Chạy `javap -c -p Foo` cho ra bytecode (slot 0 là `this`, 1 là a, 2 là b, 3 là c):

```
0: iload_1      // đẩy a lên operand stack
1: iload_2      // đẩy b
2: iadd         // pop hai số, cộng, push kết quả
3: istore_3     // pop vào biến c
4: iload_3
5: ireturn
```

Các nhóm lệnh quan trọng:

- **Load/Store**: `iload`, `aload`, `istore`, `astore`, `ldc` (nạp hằng từ constant pool).
- **Object**: `new` (cấp phát), `dup`, `getfield`, `putfield`, `getstatic`, `putstatic`, `checkcast`, `instanceof`.
- **Gọi method** (5 lệnh):
  - `invokestatic`: method static.

- `invokespecial`: constructor, `super.method()`, method private (không đa hình).
- `invokevirtual`: method instance thường (đa hình, tra vtable).
- `invokeinterface`: gọi qua interface (tra itable, chậm hơn một chút).
- `invokedynamic` (Java 7+): liên kết lúc chạy, dùng cho lambda và nối chuỗi `+` (Java 9+, qua `StringConcatFactory`).

Overload và override: overload (nạp chồng) được chọn lúc **compile** dựa trên kiểu khai báo; override (ghi đè) được chọn lúc **chạy** dựa trên kiểu thực của object (dynamic dispatch, qua bảng **vtable** của class). Field và method static **không** có tính đa hình: truy cập theo kiểu khai báo.

```
class A { String name = "A"; String who() { return "A"; } }
class B extends A { String name = "B"; String who() { return "B"; } }
A x = new B();
x.name;   // "A"  (field theo kiểu khai báo)
x.who();  // "B"  (method theo kiểu thực)
```

8.2 Object layout và synchronized bên trong

Mỗi object trên heap (HotSpot 64-bit) gồm:

```
| Object Header                            | Instance data | Padding |
| Mark Word (8 byte) | Klass pointer (4*) |   các field   | căn 8 B |
   (* 4 byte khi bật compressed class pointers, mặc định)
```

Ví dụ `new Object()` chiếm 16 byte (12 byte header + 4 byte đệm). Dùng thư viện **JOL** (Java Object Layout) để xem thực tế. Mảng có thêm 4 byte lưu độ dài.

Mark Word lưu hashCode, tuổi GC (age, 4 bit nên tối đa 15) và **trạng thái khóa**:

2 bit thấp	Trạng thái
01	Không khóa (hoặc biased lock ở bản cũ)
00	Lightweight lock: mark word trỏ tới *lock record* trên stack của thread giữ khóa
10	Heavyweight lock: mark word trỏ tới `ObjectMonitor`
11	Đã bị GC đánh dấu

Quá trình nâng cấp (inflation): không tranh chấp thì dùng lightweight lock (một lệnh CAS, rất rẻ) → có thread thứ hai tranh chấp, thread đó quay vài vòng (*adaptive spinning*) → vẫn không được thì nâng lên heavyweight, thread bị đưa vào hàng chờ và hệ điều hành chặn nó (tốn context switch). Khóa **không tự hạ cấp** ngay mà chỉ được dọn khi tới safepoint. Biased lock đã bị gỡ ở Java 18 vì chi phí duy trì lớn hơn lợi ích.

ObjectMonitor (HotSpot, C++) có các thành phần chính:

- `_owner`: thread đang giữ monitor; `_recursions`: số lần vào lại (reentrant).
- `_EntryList` / `_cxq`: các thread **BLOCKED** đang chờ giành khóa.
- `_WaitSet`: các thread gọi `wait()` (trạng thái **WAITING**).

Luồng hoạt động: `monitorenter` giành owner; `wait()` nhả monitor và vào WaitSet; `notify()` chuyển một thread từ WaitSet sang EntryList (nó vẫn phải giành lại khóa mới chạy tiếp, nên sau notify chưa chạy ngay); `monitorexit` nhả khóa và đánh thức thread kế tiếp trong EntryList. Với method synchronized, flag `ACC_SYNCHRONIZED` ở method info thay cho cặp `monitorenter/monitorexit`. Khối synchronized sinh thêm một handler exception để đảm bảo `monitorexit` luôn được gọi.

Các tối ưu của JIT: **lock elision** (bỏ khóa nếu escape analysis chứng minh object không chia sẻ), **lock coarsening** (gộp nhiều lần khóa liên tiếp trên cùng object thành một).

8.3 Reordering và memory barrier

Sắp xếp lại lệnh xảy ra ở ba tầng: **trình biên dịch/JIT**, **CPU** (out-of-order execution, store buffer), **hệ thống cache**. Mỗi core có cache và store buffer riêng nên ghi của core này chưa chắc thấy ngay ở core khác. Luật chung: sắp xếp lại được phép miễn kết quả **trong một thread** không đổi (*as-if-serial*), nhưng thread khác có thể quan sát thấy thứ tự lạ.

```
// Ban đầu x = 0, y = 0, a = 0, b = 0
// Thread 1: x = 1; a = y;      Thread 2: y = 1; b = x;
// Kết quả a == 0 và b == 0 là CÓ THỂ xảy ra khi không dùng volatile
```

Bốn loại **memory barrier** (rào chắn bộ nhớ) mà JVM chèn:

Barrier	Ý nghĩa
LoadLoad	Đọc trước hoàn tất trước đọc sau
StoreStore	Ghi trước hiển thị trước ghi sau
LoadStore	Đọc trước hoàn tất trước ghi sau

Barrier	Ý nghĩa
StoreLoad	Ghi trước hiển thị trước đọc sau (đắt nhất, "full fence")

volatile được hiện thực: trước khi ghi chèn StoreStore; sau khi ghi chèn StoreLoad; sau khi đọc chèn LoadLoad và LoadStore. Trên x86 (mô hình bộ nhớ TSO, vốn đã mạnh) chỉ cần lệnh `lock addl` cho StoreLoad; trên ARM tốn nhiều barrier hơn. Vì vậy code chạy đúng trên x86 chưa chắc đúng trên ARM nếu thiếu đồng bộ.

Safe publication (công bố object an toàn cho thread khác): (1) gán vào field `volatile` hoặc `AtomicReference`; (2) gán vào field `final` trong constructor; (3) gán trong khối `synchronized` / lấy qua cùng lock; (4) đưa qua `ConcurrentHashMap` hoặc `BlockingQueue`; (5) khởi tạo static (class initialization là thread-safe). Công bố không an toàn có thể khiến thread khác thấy object **dở dang**: field mang giá trị mặc định dù constructor đã gán.

```
class Holder { int n; Holder(int n) { this.n = n; } }
Holder h;                          // KHÔNG volatile
// Thread A: h = new Holder(42);
// Thread B: if (h != null) print(h.n);   // có thể in 0!
```

Nếu `n` là `final` thì thread B luôn thấy 42 (đảm bảo của final field), miễn không để `this` rò rỉ ra ngoài trong constructor.

8.4 GC chi tiết

Các cơ chế nền tảng

- **Tri-color marking**: object **trắng** (chưa thăm), **xám** (đã thăm nhưng con chưa duyệt hết), **đen** (đã duyệt xong). Khi đánh dấu chạy **đồng thời** với ứng dụng, ứng dụng có thể gán tham chiếu từ object đen sang object trắng làm GC xóa nhầm object còn sống, nên cần **barrier** để bắt các thay đổi tham chiếu: *SATB* (Snapshot-At-The-Beginning, G1 dùng pre-write barrier ghi lại tham chiếu cũ) hoặc *incremental update*.
- **Card table và Remembered Set**: Minor GC chỉ quét Young, nhưng object Old có thể trỏ vào Young. Để khỏi quét cả Old, JVM chia heap thành "card" 512 byte; mỗi lần ghi tham chiếu sẽ đánh dấu card "bẩn" (post-write barrier). Minor GC chỉ quét các card bẩn. G1 có Remembered Set riêng cho từng region.
- **Safepoint**: điểm mà thread có thể dừng an toàn (cuối vòng lặp, trước gọi method...). Vòng lặp đếm dài (int) không có safepoint poll có thể làm cả JVM chờ.

G1 GC: các pha

1. **Young GC** (STW, song song): dọn Eden và Survivor, sao chép object sống sang Survivor hoặc Old (*evacuation*).
2. Khi heap Old đạt ngưỡng **IHOP** (`-XX:InitiatingHeapOccupancyPercent`, mặc định 45%, G1 tự điều chỉnh) thì bắt đầu **Concurrent Marking Cycle**:
   - *Initial Mark* (STW, đi kèm một Young GC): đánh dấu root.
   - *Root Region Scan*, *Concurrent Mark* (chạy cùng ứng dụng).
   - *Remark* (STW ngắn): xử lý nốt phần SATB.
   - *Cleanup* (STW ngắn): đếm độ sống từng region, giải phóng region trống hoàn toàn.
3. **Mixed GC**: Young + một số region Old có nhiều rác nhất, lặp vài lần đến khi dưới ngưỡng rác `-XX:G1HeapWastePercent`.
4. Nếu không thu kịp (**to-space exhausted**, evacuation failure) thì rơi vào **Full GC** (đơn luồng ở bản cũ, rất chậm).

Những điều cần nhớ khi tuning G1: đừng đặt `-Xmn` cố định (làm G1 mất khả năng tự điều chỉnh để đạt mục tiêu pause); object **Humongous** (từ 50% kích thước region trở lên) cấp phát thẳng vào Old, tốn kém, giải quyết bằng tăng `-XX:G1HeapRegionSize` hoặc giảm kích thước mảng/buffer lớn; mục tiêu pause quá thấp sẽ làm giảm throughput.

ZGC và Shenandoah

- **ZGC**: dùng **colored pointers** (nhét metadata như "đã đánh dấu", "đã chuyển" vào các bit cao của con trỏ 64-bit) và **load barrier** (chạy mỗi lần đọc tham chiếu từ heap, tự sửa tham chiếu nếu object đã bị dời). Nhờ đó cả marking lẫn **relocation (compaction)** đều chạy đồng thời với ứng dụng; STW chỉ để quét root nên pause thường dưới 1ms bất kể kích thước heap. Đổi lại tốn thêm CPU và bộ nhớ. Java 21 có ZGC phân thế hệ: `-XX:+UseZGC -XX:+ZGenerational`.
- **Shenandoah**: cũng compaction đồng thời, dùng *load reference barrier*; có trong OpenJDK, không có trong Oracle JDK.

Chọn collector theo bài toán

Tình huống	Gợi ý
Container nhỏ (1 CPU, dưới 2GB), tool dòng lệnh	Serial
Job batch, ưu tiên tổng thời gian chạy	Parallel
Dịch vụ web thông thường, heap vài GB	G1 (mặc định, đa số trường hợp là đủ)

Tình huống	Gợi ý
Cần pause dưới vài ms, heap lớn	ZGC (hoặc Shenandoah)

8.5 Đọc GC log

Bật log: `-Xlog:gc*:file=gc.log:time,uptime,level,tags`. Một dòng G1 điển hình:

```
[12.345s][info][gc] GC(27) Pause Young (Normal) (G1 Evacuation Pause) 512M->128M(1024M) 15.234ms
```

Đọc: sau 12.3 giây, GC lần thứ 27, là Young GC thông thường; trước GC heap dùng 512MB, sau GC còn 128MB, tổng heap hiện tại 1024MB; pause 15.2ms. Ý nghĩa các chỉ số:

- **Allocation rate**: tốc độ cấp phát (MB/s). Cao thì Young GC xảy ra dồn dập.
- **Promotion rate**: lượng object được đẩy lên Old mỗi giây. Cao nghĩa là object sống lâu hơn dự kiến hoặc Survivor quá nhỏ (premature promotion).
- **Live set sau Full GC**: lượng dữ liệu thật sự còn sống; nếu tăng đều sau mỗi lần Full GC thì có **memory leak**.

Dấu hiệu bệnh thường gặp:

Triệu chứng trong log	Nguyên nhân thường gặp	Hướng xử lý
Full GC lặp liên tục, heap không giảm	Leak hoặc heap quá nhỏ	Heap dump + MAT; tăng heap
`To-space exhausted`	G1 không kịp thu	Giảm IHOP, tăng heap, kiểm tra Humongous
`Pause Young` kéo dài	Nhiều object sống, nhiều luồng GC thiếu CPU	Giảm lượng object sống, kiểm tra CPU limit của container
`G1 Humongous Allocation`	Mảng hoặc buffer quá lớn	Tăng region size, chia nhỏ buffer
`Metadata GC Threshold`	Nạp quá nhiều class (proxy sinh động, redeploy)	Tăng `MaxMetaspaceSize`, tìm ClassLoader leak

Công cụ trực quan: GCeasy, GCViewer, JDK Mission Control.

8.6 Bài thực hành: tái hiện các lỗi bộ nhớ

Heap OOM (chạy với `-Xmx64m -XX:+HeapDumpOnOutOfMemoryError`):

```
List<byte[]> list = new ArrayList<>();
while (true) list.add(new byte[1024 * 1024]);   // OutOfMemoryError: Java
heap space
```

StackOverflowError (chạy với `-Xss256k` để thấy nhanh hơn):

```
static int depth = 0;
static void recurse() { depth++; recurse(); }
// bắt StackOverflowError rồi in depth để xem độ sâu tối đa
```

Metaspace OOM (`-XX:MaxMetaspaceSize=32m`): dùng ByteBuddy hoặc `Proxy` sinh class liên tục với ClassLoader mới mỗi lần.

Direct memory OOM (`-XX:MaxDirectMemorySize=32m`): gọi liên tục `ByteBuffer.allocateDirect(1024 * 1024)` và giữ tham chiếu.

Việc cần làm sau mỗi lần: mở heap dump bằng MAT, xem *Dominator Tree* (object nào giữ nhiều bộ nhớ nhất), *Path to GC Roots* (vì sao object không bị thu); đối chiếu với GC log và `jstat -gcutil`.

PHẦN 9. ĐI SÂU VÀO CONCURRENCY VÀ COLLECTION

9.1 AQS (AbstractQueuedSynchronizer)

AQS là khung xương của `ReentrantLock`, `Semaphore`, `CountDownLatch`, `ReentrantReadWriteLock`, `FutureTask`... Hiểu AQS là hiểu hầu hết `java.util.concurrent`.

Hai thành phần:

1. Một biến `volatile int state` thể hiện trạng thái đồng bộ (ý nghĩa tùy lớp con): với ReentrantLock là số lần giữ khóa (0 = rảnh); với Semaphore là số permit còn lại; với CountDownLatch là bộ đếm.
2. Một **hàng đợi FIFO hai chiều** (biến thể CLH) chứa các thread đang chờ, mỗi thread là một `Node`.

Quy trình `lock()` kiểu độc quyền (exclusive):

```
acquire(): tryAcquire() thành công? -> xong
           không -> addWaiter(): tạo Node, CAS đưa vào đuôi hàng đợi
                 -> acquireQueued(): vòng lặp
                      nếu Node là kế tiếp sau head và tryAcquire() ok ->
thành head, xong
                      không -> park() thread (LockSupport.park), chờ được
đánh thức
release(): tryRelease() ok -> unparkSuccessor(head): đánh thức Node kế tiếp
```

Lớp con chỉ cần viết `tryAcquire/tryRelease` (chế độ độc quyền) hoặc `tryAcquireShared/tryReleaseShared` (chế độ chia sẻ), phần xếp hàng, ngủ, đánh thức do AQS lo. Ví dụ khóa không reentrant tối giản:

```
class Mutex {
    private static class Sync extends AbstractQueuedSynchronizer {
        protected boolean tryAcquire(int n) { return compareAndSetState(0, 1)
&& setOwner(); }
        protected boolean tryRelease(int n) { setExclusiveOwnerThread(null);
setState(0); return true; }
        private boolean setOwner() {
setExclusiveOwnerThread(Thread.currentThread()); return true; }
    }
    private final Sync sync = new Sync();
    public void lock()   { sync.acquire(1); }
    public void unlock() { sync.release(1); }
}
```

Fair và non-fair trong ReentrantLock: non-fair (mặc định) cho thread vừa gọi `lock()` thử CAS giành khóa ngay (*barging*) dù có người đang xếp hàng, nên throughput cao hơn nhưng có nguy cơ starvation; fair kiểm tra `hasQueuedPredecessors()` và luôn xếp hàng. **Condition** (`await/signal`) có hàng đợi riêng cho mỗi Condition: `await` nhả khóa và chuyển Node sang hàng đợi condition, `signal` chuyển Node ngược lại hàng đợi khóa.

CountDownLatch dùng chế độ chia sẻ: `state` = N; `countDown()` giảm state; `await()` chặn đến khi state về 0, lúc đó đánh thức **mọi** thread đang chờ (lan truyền kiểu *propagate*).

9.2 ThreadPoolExecutor bên trong

Trạng thái đóng gói trong một biến: `AtomicInteger ctl` chứa 3 bit cao là `runState`, 29 bit thấp là `workerCount` (tối đa khoảng 536 triệu worker). Thay đổi cả hai nguyên tử bằng một lần CAS.

Trạng thái	Ý nghĩa
RUNNING	Nhận task mới, xử lý task trong hàng đợi
SHUTDOWN	Gọi `shutdown()`: không nhận task mới, vẫn xử lý nốt hàng đợi
STOP	Gọi `shutdownNow()`: không nhận mới, không xử lý hàng đợi, ngắt (interrupt) các worker
TIDYING	Mọi worker đã dừng, chuẩn bị gọi hook `terminated()`
TERMINATED	Hoàn tất

Luồng `execute(task)`:

1. `workerCount < corePoolSize` → `addWorker(task, core=true)` tạo thread mới chạy task đó ngay.
2. Ngược lại, `workQueue.offer(task)`: thành công thì (kiểm tra lại trạng thái; nếu pool không còn worker nào thì thêm một worker không có task).
3. Hàng đợi đầy → `addWorker(task, core=false)` nếu `workerCount < maximumPoolSize`.
4. Không thêm được → `RejectedExecutionHandler`.

Worker: mỗi worker là một object vừa implement `Runnable` vừa kế thừa AQS (dùng như khóa độc quyền để biết worker đang rảnh hay đang chạy task, phục vụ `interruptIdleWorkers`). Vòng đời của thread worker là `runWorker()`: lặp `while (task != null || (task = getTask()) != null)` chạy task. `getTask()` lấy từ hàng đợi bằng `take()` (chờ vô hạn) hoặc `poll(keepAliveTime)` khi pool đang có nhiều thread hơn core (hoặc bật `allowCoreThreadTimeOut`); poll hết thời gian thì worker thoát và bị hủy.

Xử lý exception: nếu task ném exception qua `execute()`, exception đi ra `UncaughtExceptionHandler`, thread worker **chết** và pool tạo worker thay thế. Nếu dùng `submit()`, exception bị **nuốt vào Future** và chỉ lộ ra khi gọi `get()` (rất hay bị bỏ sót lỗi!). Cách xử lý: bọc task bằng try/catch ghi log, hoặc override `afterExecute`.

Cạm bẫy thường gặp:

- Hàng đợi không giới hạn làm `maximumPoolSize` **không bao giờ có tác dụng** (vì hàng đợi không bao giờ "đầy" để kích hoạt bước 3).
- Quên `shutdown()` thì thread non-daemon giữ JVM sống mãi.
- Task chờ lẫn nhau trong cùng một pool nhỏ gây **thread starvation deadlock** (task cha chờ task con nhưng mọi worker đều bị task cha chiếm).
- Giám sát qua `getPoolSize()`, `getActiveCount()`, `getQueue().size()`, `getCompletedTaskCount()`; luôn đặt tên thread bằng `ThreadFactory`.

9.3 HashMap bên trong (Java 8+)

Cấu trúc: mảng `Node<K,V>[] table`, mỗi ô (bucket) là danh sách liên kết, chuyển thành **cây đỏ đen** khi bucket có từ 8 node trở lên *và* mảng đã có từ 64 ô (nếu mảng còn nhỏ thì resize thay vì treeify); cây quay lại danh sách khi còn 6 node trở xuống.

Tính vị trí:

```
static int hash(Object key) {
    int h = key.hashCode();
    return h ^ (h >>> 16);          // trộn 16 bit cao xuống thấp để giảm
đụng độ
}
int index = (table.length - 1) & hash;   // nhanh hơn %, đúng vì length là
lũy thừa của 2
```

Luồng `put(k, v)`: tính hash → ô rỗng thì tạo node → ô có node thì duyệt (hoặc tìm trong cây): trùng key (`hash` bằng và `==` hoặc `equals`) thì ghi đè value, không thì thêm vào cuối → đủ ngưỡng thì treeify → `++size > threshold` thì **resize**.

Resize: mảng tăng gấp đôi, `threshold = capacity * loadFactor` (mặc định 0.75). Nhờ kích thước là lũy thừa 2, mỗi node chỉ có hai khả năng: giữ nguyên chỉ số cũ, hoặc dời sang `chỉ số cũ + oldCap`, tùy một bit của hash, nên chia bucket thành hai nửa (lo và hi) mà **không cần tính lại hash** và giữ nguyên thứ tự. Java 7 chèn đầu danh sách và đảo thứ tự khi resize, làm xuất hiện vòng lặp vô hạn khi nhiều thread cùng resize; Java 8 sửa phần thứ tự nhưng `HashMap` vẫn **không thread-safe** (mất dữ liệu, size sai).

Sức chứa ban đầu: nếu biết trước N phần tử, khởi tạo `new HashMap<>((int) (N / 0.75f) + 1)` để tránh resize nhiều lần. `HashMap(int cap)` làm tròn lên lũy thừa của 2 gần nhất.

Hợp đồng equals và hashCode: hai object `equals` bắt buộc cùng `hashCode`; ngược lại không bắt buộc. `hashCode` tệ (ví dụ luôn trả hằng số) khiến mọi phần tử dồn vào một bucket, thao tác tụt từ O(1) xuống O(n) (hoặc O(log n) nếu đã treeify và key `Comparable`). Key nên bất biến; nếu thay đổi field tham gia `hashCode` sau khi `put` thì không tìm lại được entry.

9.4 ConcurrentHashMap bên trong (Java 8+)

Cùng cấu trúc với HashMap (mảng + danh sách/cây) nhưng dùng **CAS + khóa từng bucket** thay cho cơ chế `Segment` của Java 7.

- **Khởi tạo bảng**: thread nào CAS thành công `sizeCtl` thì khởi tạo, các thread khác `yield` chờ.
- **`putVal`**: ô rỗng → `casTabAt` (CAS đặt node đầu tiên, **không khóa**). Ô đang có node → `synchronized (nodeĐầu)`: chỉ khóa **một bucket**, các bucket khác vẫn ghi song song. Nếu bucket đang được di chuyển (node `ForwardingNode`) thì thread gọi `helpTransfer` **cùng giúp resize**.
- **`get`**: **không khóa** (field `val` và `next` là `volatile`), nên đọc rất nhanh.
- **Resize đồng thời**: nhiều thread chia nhau từng đoạn bảng để chuyển sang bảng mới.
- **`size()`**: không khóa toàn bảng mà cộng `baseCount` với các ô `CounterCell` (cùng ý tưởng `LongAdder` để giảm tranh chấp), nên là giá trị ước lượng tại thời điểm gọi.
- **Vì sao không cho null**: nếu `get(k)` trả `null` thì không phân biệt được "không có key" hay "value là null", và trong môi trường đa luồng không thể gọi thêm `containsKey` để phân biệt vì giữa hai lời gọi map có thể đã đổi.

Hàm nguyên tử theo key: `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`. Lưu ý: hàm truyền vào `compute*` chạy **trong khi giữ khóa bucket**, nên phải ngắn, không chặn I/O, và **không được sửa chính map đó** (có thể deadlock hoặc ném lỗi).

```
// SAI: check-then-act, hai thread cùng thấy chưa có
if (!map.containsKey(k)) map.put(k, new Expensive());
// ĐÚNG: nguyên tử
map.computeIfAbsent(k, key -> new Expensive());
// Đếm từ: nguyên tử
counts.merge(word, 1, Integer::sum);
```

9.5 Đọc thread dump và bài thực hành deadlock

Tạo deadlock:

```
Object a = new Object(), b = new Object();
new Thread(() -> { synchronized (a) { sleep(100); synchronized (b) { } } },
"T1").start();
new Thread(() -> { synchronized (b) { sleep(100); synchronized (a) { } } },
"T2").start();
```

Chạy `jps` lấy PID rồi `jstack PID` (hoặc `jcmd PID Thread.print`). Đoạn đáng chú ý:

```
"T1" #21 prio=5 tid=0x... nid=0x1a2b waiting for monitor entry
   java.lang.Thread.State: BLOCKED (on object monitor)
        at Demo.lambda$main$0(Demo.java:5)
        - waiting to lock <0x000000076ab62208> (a java.lang.Object)
        - locked <0x000000076ab62218> (a java.lang.Object)

Found one Java-level deadlock:
=============================
"T1": waiting to lock monitor 0x..., which is held by "T2"
"T2": waiting to lock monitor 0x..., which is held by "T1"
```

Cách đọc: mỗi thread có tên, trạng thái (`RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`) và stack. `waiting to lock <addr>` nghĩa là đang chờ khóa; `locked <addr>` là khóa đang giữ; `parking to wait for <addr>` là chờ qua `LockSupport` (Lock, Condition, BlockingQueue); `waiting on <addr>` là `Object.wait()`. `nid` là mã thread ở hệ điều hành dạng hex, dùng khớp với `top -H` khi điều tra CPU cao.

Các mẫu bệnh thường gặp:

Dấu hiệu trong dump	Chẩn đoán
Nhiều thread BLOCKED cùng chờ một địa chỉ	Tranh chấp một khóa (lock contention): thu hẹp phạm vi khóa, chia nhỏ khóa
Mọi worker của pool WAITING trong `getTask()`	Pool rảnh, bình thường
Mọi worker WAITING ở `Future.get()`	Thread starvation deadlock (task chờ task trong cùng pool)
Nhiều thread RUNNABLE tại cùng một dòng code	Điểm nóng CPU (vòng lặp, regex xấu, hash đụng độ)
Thread bị kẹt ở `SocketInputStream.socketRead`	Gọi mạng không đặt timeout

Nên chụp **3 lần cách nhau vài giây** và so sánh: thread nào luôn nằm ở cùng chỗ là thủ phạm.

9.6 Mô hình Producer-Consumer hoàn chỉnh

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(100);
final int POISON = -1;                        // tín hiệu kết thúc (poison
pill)

Thread producer = new Thread(() -> {
    try {
        for (int i = 0; i < 1000; i++) q.put(i);   // đầy thì tự chặn (back-
pressure)
        q.put(POISON);
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

Thread consumer = new Thread(() -> {
    try {
        while (true) {
            int v = q.take();                      // rỗng thì tự chặn
            if (v == POISON) break;
            process(v);
```

```
        }
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
producer.start(); consumer.start();
```

Hàng đợi **có giới hạn** tạo cơ chế *back-pressure*: producer nhanh hơn consumer thì bị chặn, tránh tràn bộ nhớ. Nhiều consumer thì gửi mỗi consumer một poison pill.

9.7 Hiệu năng đa luồng: những điều phản trực giác

- **Định luật Amdahl**: tăng tốc tối đa = 1 / (S + (1 - S) / N), với S là phần tuần tự. Nếu 10% công việc phải tuần tự thì dù có vô hạn core cũng chỉ nhanh tối đa 10 lần.
- **False sharing**: hai biến độc lập nằm cùng một cache line (64 byte) bị hai core ghi liên tục làm cache line bị vô hiệu hóa qua lại. Khắc phục bằng đệm (padding) hoặc `@jdk.internal.vm.annotation.Contended`; `LongAdder` dùng kỹ thuật này.
- **Lock contention** thường đắt hơn bản thân phép tính: giải pháp là thu hẹp vùng khóa, *lock striping* (chia nhỏ khóa), dữ liệu bất biến, hoặc cấu trúc lock-free.
- **Context switch** và quá nhiều thread làm chậm hệ thống; số thread không nên vượt xa số core với việc nặng CPU.
- Đừng tối ưu theo cảm tính: đo bằng **JMH**, profiling bằng JFR hoặc async-profiler.

```
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5) @Measurement(iterations = 5) @Fork(2)
public class MyBench {
    @Benchmark
    public int test(Blackhole bh) {        // Blackhole chặn JIT xóa mã "vô
dụng"
        return compute();
    }
}
```

9.8 Bộ bài tập thực hành đề xuất

1. Viết chương trình đếm đến 1.000.000 bằng 10 thread với biến `int`, `volatile int`, `synchronized`, `AtomicInteger`, `LongAdder`; so sánh kết quả đúng sai và thời gian bằng JMH.
2. Tái hiện lỗi hiển thị: vòng lặp `while (!flag)` không thoát khi `flag` không volatile (chạy với `-server`); sửa bằng volatile.

3. Viết lại `Mutex` ở mục 9.1 thành khóa reentrant, thêm `Condition`.
4. Tự cài một thread pool tối giản (hàng đợi + worker + `shutdown`), sau đó đối chiếu với `ThreadPoolExecutor`.
5. Tạo deadlock, bắt bằng `jstack`, rồi sửa bằng cách khóa theo thứ tự cố định và bằng `tryLock(timeout)`.
6. Gây rò rỉ bằng `ThreadLocal` trong thread pool, chụp heap dump, tìm nguyên nhân bằng MAT.
7. Viết custom ClassLoader nạp class từ thư mục bất kỳ, chứng minh hai loader nạp cùng một class cho ra hai kiểu khác nhau (`ClassCastException`).
8. Dùng JOL in layout của vài object (có và không có field `long`, mảng) để kiểm tra lý thuyết ở mục 8.2.
9. Chạy một ứng dụng web nhỏ với G1 và ZGC, tạo tải, so sánh GC log và độ trễ p99.
10. Cài `HashMap` đơn giản (mảng + danh sách, resize khi đạt load factor), rồi mở rộng thành bản có khóa theo bucket.

PHẦN MỞ RỘNG 2: NGÔN NGỮ, THƯ VIỆN VÀ BÀI TẬP KIỂM CHỨNG

Các Phần 10 đến 15 bổ sung những mảng còn thiếu: trình biên dịch, kiểu dữ liệu, đối tượng, exception, collection, stream, I/O, mạng và runtime nâng cao. **Mỗi phần kết thúc bằng bài tập** (dự đoán kết quả, thực nghiệm, lập trình) kèm đáp án gợi ý. Phần 16 gồm bài tập cho các Phần 1 đến 9 và các dự án tổng hợp. Cách học hiệu quả nhất: **tự dự đoán kết quả trước, chạy thử, rồi mới đọc đáp án**; chỗ nào sai thì quay lại phần lý thuyết.

PHẦN 10. TRÌNH BIÊN DỊCH, KIỂU DỮ LIỆU VÀ CHUỖI

10.1 javac làm gì với mã nguồn của bạn

Các bước chính: **parse** (tạo cây cú pháp) → **enter** (đăng ký class, member) → **attribute** (suy luận và kiểm tra kiểu, giải quyết overload) → **flow** (kiểm tra gán biến, return, checked exception) → **desugar** (biến cú pháp tiện lợi thành mã đơn giản) → **generate** (sinh bytecode). Phần desugar giải thích nhiều hành vi "khó hiểu":

Cú pháp	Thực chất javac sinh ra
for-each trên mảng	Vòng lặp for thường với chỉ số
for-each trên Iterable	`Iterator`, gọi `hasNext/next`
Autoboxing	`Integer.valueOf(x)` và `intValue()`
switch trên String	`hashCode()` để rẽ nhánh, rồi `equals` để xác nhận
switch trên enum	Mảng phụ `$SwitchMap$...` ánh xạ ordinal sang số nhánh
try-with-resources	try/finally, `close()` và `addSuppressed`
Lambda	`invokedynamic` + method private synthetic `lambda$main$0`
Inner class (không static)	Field synthetic `this$0` trỏ tới outer
Generics	Xóa kiểu, thêm `checkcast`, sinh **bridge method**
Varargs	Mảng tạo ngầm ở nơi gọi
`assert`	Field `$assertionsDisabled`, kiểm tra lúc chạy
Enum	Class kế thừa `Enum`, `values()`, mảng `$VALUES`
Record	Class final, field private final, `toString/equals/hashCode` qua `ObjectMethods` (invokedynamic)

Lambda dưới nắp máy: lần chạy đầu, `invokedynamic` gọi `LambdaMetafactory` để sinh động một class implement functional interface. Lambda **không capture** biến nào chỉ tạo một instance duy nhất dùng lại; lambda **có capture** tạo instance mới mỗi lần đánh giá. Lambda không phải inner class: `this` bên trong lambda là `this` của class bao ngoài, và lambda không tạo ra file .class riêng.

```
Runnable r1 = () -> System.out.println("a");   // không capture
int n = 5;
Runnable r2 = () -> System.out.println(n);     // capture n
// Gọi hai lần cùng một dòng code: r1 luôn là cùng object, r2 là object mới
```

Hằng số compile-time và constant folding: biểu thức gồm literal và biến `final` đã gán hằng được tính ngay lúc compile và đưa vào constant pool, nên tham gia String Pool.

```
final String a = "ja";
String b = a + "va";        // folding thành "java"
System.out.println(b == "java");   // true
String c = "ja";
System.out.println((c + "va") == "java");   // false: c không final, nối lúc
chạy
```

10.2 Số nguyên và số thực

Số nguyên

- Tràn số xảy ra **im lặng**: `Integer.MAX_VALUE + 1 == Integer.MIN_VALUE`. Dùng `Math.addExact/multiplyExact/toIntExact` để ném `ArithmeticException`.
- `Math.abs(Integer.MIN_VALUE)` vẫn âm (không có số dương tương ứng).
- Chia và dư lấy dấu theo số bị chia: `-7 / 2 == -3`, `-7 % 2 == -1`. Muốn dư không âm dùng `Math.floorMod(-7, 2) == 1`.
- Toán tử dịch bit: `>>` giữ dấu, `>>>` điền 0. Số bit dịch bị che (mask) theo kiểu: với int chỉ lấy 5 bit thấp, nên `1 << 33 == 2`; với long lấy 6 bit.
- Phép toán trên `byte/short/char` luôn được **thăng kiểu lên int** trước. Vì vậy `short s = 1; s = s + 1;` lỗi compile, nhưng `s += 1;` hợp lệ do toán tử gán kép có **ép kiểu ngầm**: `byte b = 10; b += 300;` cho `b = 54`.

Số thực (IEEE 754)

Kiểu	Bit	Dấu / mũ / định trị	Độ chính xác thập phân
float	32	1 / 8 / 23	khoảng 7 chữ số
double	64	1 / 11 / 52	khoảng 15 đến 16 chữ số

- Phần lớn số thập phân (0.1, 0.2) **không biểu diễn chính xác** bằng nhị phân, nên `0.1 + 0.2 != 0.3`. So sánh bằng ngưỡng sai số `Math.abs(a - b) < eps`.
- `NaN != NaN` (kể cả với chính nó), nhưng `Double.valueOf(NaN).equals(NaN)` là true. `0.0 == -0.0` là true, nhưng `Double.valueOf(0.0).equals(-0.0)` là false. Dùng `Double.compare` để có thứ tự toàn phần.
- Chia số thực cho 0 ra `Infinity` hoặc `NaN`, không ném exception (số nguyên chia 0 mới ném `ArithmeticException`).
- Từ Java 17 mọi phép tính số thực đều theo `strictfp` (từ khóa này không còn tác dụng).

- Tiền tệ và số cần chính xác thập phân: dùng `BigDecimal` khởi tạo từ **chuỗi** hoặc `BigDecimal.valueOf(double)`. `new BigDecimal(0.1)` mang theo sai số nhị phân của 0.1. So sánh giá trị dùng `compareTo` (`equals` còn xét cả scale: `2.0` khác `2.00`). Chia có thể vô hạn nên phải truyền `RoundingMode` (HALF_EVEN là "làm tròn ngân hàng").

Ép kiểu và thăng kiểu trong biểu thức điều kiện

```
Object o = true ? 1 : 2.0;        // in ra 1.0 (nhánh bị thăng lên double)
Integer x = null;
int y = flag ? x : 0;             // NullPointerException do unboxing x
```

10.3 Boxing, unboxing và overload

- `Integer.valueOf` có **cache** cho -128 đến 127 (mở rộng cận trên bằng `-XX:AutoBoxCacheMax=N`). Vì vậy `==` trên Integer cho kết quả khác nhau theo giá trị. Luôn dùng `equals`.
- Boxing tạo object: vòng lặp tính tổng với `Long sum` chậm hơn nhiều so với `long sum` vì tạo hàng triệu object tạm.
- `List<Integer>.remove(int index)` khác `remove(Object o)`: `list.remove(1)` xóa phần tử **ở chỉ số 1**, muốn xóa giá trị 1 dùng `list.remove(Integer.valueOf(1))`.
- Chọn overload theo ba pha: (1) chỉ dùng mở rộng kiểu primitive/ tham chiếu, không boxing; (2) cho phép boxing/unboxing; (3) cho phép varargs. Ví dụ `m(long)` thắng `m(Integer)` khi gọi `m(5)`, vì pha 1 đã khớp bằng widening.

10.4 String bên trong

- Từ Java 9, String lưu bằng `byte[] value` kèm cờ `coder` (**Compact Strings**): chuỗi chỉ có ký tự Latin-1 dùng 1 byte/ký tự, còn lại dùng UTF-16 (2 byte). Tắt bằng `-XX:-CompactStrings`.
- `hashCode` được **cache** trong field `hash`; String bất biến nên an toàn khi chia sẻ giữa thread, làm key HashMap, và cho phép String Pool.
- `substring` từ Java 7u6 **sao chép** mảng (trước đó dùng chung mảng gây rò rỉ bộ nhớ).
- **char không phải ký tự**: Java dùng UTF-16, ký tự ngoài BMP (emoji, một số chữ Hán) chiếm **2 char** (surrogate pair). Do đó `"😀".length() == 2` nhưng `codePointCount(0, len) == 1`. Dùng `codePoints()` khi cần đếm hoặc duyệt đúng ký tự.
- `StringBuilder` tăng dung lượng theo công thức xấp xỉ `(cũ + 1) * 2`; nếu biết trước kích thước nên đặt dung lượng ban đầu. `StringBuffer` có đồng bộ nên chậm hơn, hiếm khi cần.

- `split` nhận **regex** (ký tự đặc biệt như `|`, `.` phải escape) và bỏ các chuỗi rỗng ở cuối. Chuỗi nhiều dòng dùng text block `"""`.
- Charset: Java 18+ mặc định đọc ghi theo UTF-8 (JEP 400). Mã cũ phụ thuộc charset mặc định của hệ điều hành dễ lỗi font; luôn truyền `StandardCharsets.UTF_8` rõ ràng.
- Nối chuỗi bằng `+` trong **vòng lặp** tạo nhiều object trung gian ($O(n^2)$); nối ngoài vòng lặp thì javac tối ưu (invokedynamic từ Java 9). Với ghép nhiều phần tử dùng `StringBuilder` hoặc `String.join` / `Collectors.joining`.

10.5 Bài tập Phần 10

A. Dự đoán kết quả (không chạy trước)

1. `Integer a = 127, b = 127, c = 128, d = 128; System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));`
2. Với `s1 = "java"`, `s2 = "ja" + "va"`, `s3 = new String("java")`, `final String p = "ja"; s4 = p + "va"`, `String q = "ja"; s5 = q + "va"`: các biểu thức `s1==s2`, `s1==s3`, `s1==s4`, `s1==s5`, `s1==s5.intern()` cho kết quả gì?
3. `byte b = 10; b += 300; System.out.println(b);`
4. `System.out.println(0.1 + 0.2 == 0.3); System.out.println(Double.NaN == Double.NaN); System.out.println(0.0 == -0.0); System.out.println(Double.valueOf(0.0).equals(-0.0));`
5. `System.out.println(-7 / 2); System.out.println(-7 % 2); System.out.println(Math.floorMod(-7, 2)); System.out.println(1 << 33); System.out.println(-1 >>> 28);`
6. `List<Integer> list = new ArrayList<>(List.of(1, 2, 3)); list.remove(1); System.out.println(list);`
7. `Object o = true ? 1 : 2.0; System.out.println(o);`

Đáp án gợi ý: (1) `true false true`. (2) `true`, `false`, `true` (p là hằng nên được folding), `false`, `true`. (3) `54`, vì 310 mod 256 = 54. (4) `false`, `false`, `true`, `false`. (5) `-3`, `-1`, `1`, `2`, `15`. (6) `[1, 3]` (xóa theo chỉ số). (7) `1.0`.

B. Thực nghiệm

8. Viết hai lambda (có và không capture biến), chạy `javap -c -p`, tìm lệnh `invokedynamic` và method `lambda$...`. Gọi mỗi lambda hai lần trong cùng một dòng code (trong vòng lặp) rồi so sánh `==` giữa hai object nhận được. Giải thích kết quả.

9. Dùng JMH so sánh ba cách nối 1000 chuỗi: `+` trong vòng lặp, `StringBuilder`, `String.join`. Ghi lại thời gian và số object cấp phát (`-prof gc`).
10. Viết method đếm số ký tự thật của một chuỗi chứa emoji và chữ Hán bằng `length()` và `codePointCount`; kiểm tra khác nhau ở đâu.
11. Dùng JOL, so sánh kích thước chuỗi `"hello"` khi chạy với `-XX:+CompactStrings` và `-XX:-CompactStrings`.
12. Chứng minh `new BigDecimal(0.1)` khác `BigDecimal.valueOf(0.1)` bằng cách in cả hai; giải thích.

C. Lập trình

13. Viết `safeAdd(int a, int b)` dùng `Math.addExact` và bắt tràn số, trả về `OptionalInt`.
14. Viết `Money` bất biến (record) dựa trên `BigDecimal`, cung cấp `plus`, `times`, `allocate(int parts)` chia tiền thành các phần mà **tổng luôn đúng** (không mất một đồng lẻ).

PHẦN 11. LỚP, ĐỐI TƯỢNG VÀ EXCEPTION CHUYÊN SÂU

11.1 Thứ tự khởi tạo

Thứ tự: static của cha → static của con (chỉ **một lần**, lúc class được khởi tạo) → với mỗi lần `new`: instance initializer và field của cha, constructor cha → instance initializer và field của con, constructor con. Field và initializer được thực thi **theo thứ tự xuất hiện** trong mã.

Cạm bẫy kinh điển: **gọi method có thể bị ghi đè trong constructor**. Khi constructor cha chạy, field của con **chưa được gán** (đang mang giá trị mặc định).

```
class P {
    static { System.out.println("P static"); }
    { System.out.println("P instance"); }
    P() { System.out.println("P ctor"); show(); }
    void show() { System.out.println("P show"); }
}
class C extends P {
    static { System.out.println("C static"); }
    int x = 5;
    { System.out.println("C instance"); }
    C() { System.out.println("C ctor x=" + x); }
    @Override void show() { System.out.println("C show x=" + x); }
}
// new C();
```

11.2 Các loại class lồng nhau

Loại	Giữ tham chiếu outer?	Ghi chú
static nested	Không	Nên là mặc định khi class lồng không cần outer
inner (member)	Có (`this$0`)	Rò rỉ bộ nhớ nếu inner sống lâu hơn outer
local class	Có (nếu ở ngữ cảnh instance)	Capture biến cục bộ effectively final
anonymous	Có (nếu ở ngữ cảnh instance)	Một ví dụ rò rỉ: Handler hoặc listener ẩn danh gắn vào đối tượng sống lâu

Biến cục bộ được capture phải **effectively final** vì giá trị được **sao chép** vào field synthetic của object; cho phép thay đổi sẽ làm bản sao và biến gốc lệch nhau.

11.3 Interface hiện đại và đa kế thừa hành vi

Interface có thể có `default`, `static` và `private` method (Java 8, 9). Khi một class kế thừa cùng một method từ nhiều nguồn, quy tắc giải quyết:

1. **Method của class (kể cả của cha) luôn thắng** method default của interface.
2. Nếu không, interface **cụ thể hơn** (kế thừa interface kia) thắng.

3. Nếu vẫn mơ hồ, phải **ghi đè** và có thể chọn tường minh: `A.super.hello()`.

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B {
    public String hello() { return A.super.hello() + B.super.hello(); }  //
bắt buộc ghi đè
}
```

Interface chỉ có đúng một abstract method gọi là functional interface (nên đánh dấu `@FunctionalInterface`). Field trong interface luôn là `public static final`.

11.4 Enum

Enum là class final kế thừa `java.lang.Enum`, mỗi hằng là một instance static được tạo khi class khởi tạo. Có thể có field, constructor (private), method, và **method abstract riêng cho từng hằng**:

```
enum Op {
    ADD("+") { int apply(int a, int b) { return a + b; } },
    MUL("*") { int apply(int a, int b) { return a * b; } };
    final String sym;
    Op(String sym) { this.sym = sym; }
    abstract int apply(int a, int b);
}
```

- So sánh enum bằng `==` an toàn. Enum **tự an toàn với serialization** (giải tuần tự hóa trả về đúng hằng cũ) và chống reflection tạo instance, vì vậy **enum singleton** là cách làm singleton bền nhất.
- `EnumSet` và `EnumMap` dùng vector bit và mảng theo `ordinal()`, nhanh và gọn hơn `HashSet/HashMap`. Không dựa vào `ordinal()` để lưu trữ lâu dài (đổi thứ tự khai báo sẽ vỡ dữ liệu), hãy lưu tên hoặc mã riêng.

11.5 Exception chuyên sâu

- **Phân loại**: `Error` (lỗi JVM, thường không bắt), `RuntimeException` (unchecked, lỗi lập trình), còn lại là checked. Nguyên tắc thiết kế: checked exception cho tình huống người gọi **có thể phục hồi**, unchecked cho vi phạm hợp đồng/lập trình. Nhiều thư viện hiện đại (Spring) ưu tiên unchecked để tránh `throws` lan tràn.

- **try-with-resources**: tài nguyên đóng theo thứ tự **ngược** thứ tự khai báo. Nếu thân try ném exception và `close()` cũng ném, exception của `close()` được gắn vào exception chính qua `addSuppressed` (xem `getSuppressed()`), nên không mất nguyên nhân gốc (khác với `finally` thủ công, exception của finally **ghi đè** exception gốc).
- **finally** luôn chạy trừ khi JVM dừng (`System.exit`, crash, vòng lặp vô hạn trước đó). `return` trong finally **nuốt** exception và ghi đè giá trị trả về.
- **Chi phí**: tạo exception tốn kém chủ yếu vì `fillInStackTrace()` duyệt toàn bộ stack. Với exception dùng cho điều khiển luồng trong đường nóng, có thể tắt bằng constructor `(message, cause, enableSuppression, writableStackTrace=false)`. Tuy vậy, không dùng exception thay cho điều kiện thông thường.
- `catch (Exception | Error e)` multi-catch: biến `e` ngầm là `final`. **Precise rethrow**: `catch (Exception e) { throw e; }` trong method chỉ khai báo các checked exception mà try thực sự có thể ném.
- Exception trong khối **static initializer** biến thành `ExceptionInInitializerError`; những lần dùng class sau đó nhận `NoClassDefFoundError`.
- Luôn giữ nguyên nhân gốc khi bọc: `throw new MyEx("...", cause)`. Ghi log **hoặc** ném lại, không làm cả hai (log trùng lặp).
- Không bắt `Throwable` / `Error` bừa bãi; riêng `InterruptedException` phải khôi phục cờ ngắt.

11.6 Hợp đồng của Object

- `equals`: phản xạ, đối xứng, bắc cầu, nhất quán, `x.equals(null)` là false. Kế thừa class có `equals` rồi thêm field rất dễ phá tính đối xứng/bắc cầu; ưu tiên **composition** hoặc `record`.
- `hashCode`: nhất quán với `equals`. `compareTo` nên nhất quán với `equals` (nếu không, `TreeSet` và `HashSet` cho kết quả khác nhau cùng dữ liệu, ví dụ `BigDecimal` 2.0 và 2.00).
- `clone()`: hợp đồng yếu, mặc định **shallow copy**, dễ lỗi với field mutable. Ưu tiên copy constructor hoặc factory method.
- **Công thức class bất biến**: `final` class, field `private final`, không setter, **copy phòng thủ** cho field mutable ở constructor và getter, không để `this` thoát trong constructor.
- **Covariant return** cho phép method ghi đè trả về kiểu con. Method `static` và `private` **không** có đa hình (chỉ bị *ẩn*, không bị *ghi đè*).

11.7 Bài tập Phần 11

A. Dự đoán kết quả

1. Chạy đoạn `new C()` ở mục 11.1: thứ tự in ra là gì? Giá trị `x` tại dòng `C show` là bao nhiêu?
2. Ba method sau trả về gì?

```
static int f() { try { return 1; } finally { System.out.println("f"); } }
static int g() { int i = 1; try { return i; } finally { i = 2; } }
static int h() { try { throw new RuntimeException(); } finally { return 3; }
}
```

3. Với class `C implements A, B` ở mục 11.3 nếu **không** ghi đè `hello()` thì chuyện gì xảy ra?
4. `class X { static void m() { System.out.println("X"); } } class Y extends X { static void m() { System.out.println("Y"); } } X x = new Y(); x.m();` in ra gì?

Đáp án gợi ý: (1) `P static`, `C static`, `P instance`, `P ctor`, `C show x=0`, `C instance`, `C ctor x=5`; ở dòng `C show`, x là 0 vì field con chưa được gán. (2) `f` in ra rồi trả `1`; `g` trả `1` (giá trị return đã được chốt trước finally); `h` trả `3` và exception bị nuốt. (3) Lỗi biên dịch do mơ hồ. (4) `X` (method static theo kiểu khai báo).

B. Thực nghiệm

5. Cố tình tạo rò rỉ do inner class: giữ một danh sách static các anonymous class tạo từ một outer object lớn (có mảng 10MB). Chụp heap dump, dùng MAT tìm `this$0` giữ outer. Sửa bằng static nested class.
6. Viết hai cách đóng tài nguyên: `finally` thủ công và try-with-resources, với `close()` cố tình ném exception. So sánh exception nào bị mất, exception nào được giữ và in `getSuppressed()`.
7. Đo chi phí tạo exception: JMH so sánh `new Exception()` sâu 50 frame với phiên bản `writableStackTrace=false`. Rút ra kết luận.
8. Chứng minh enum singleton chống được serialization và reflection, còn singleton thường bị phá (tạo instance thứ hai bằng reflection hoặc deserialize).

C. Lập trình

9. Cài class `Point3D` bất biến với `equals/hashCode/compareTo` đúng hợp đồng; viết test chứng minh tính đối xứng và bắc cầu. Thử kế thừa thêm field `color` và chỉ ra chỗ phá hợp đồng.
10. Viết enum `State` có `canTransitionTo(State)` biểu diễn máy trạng thái đơn hàng, dùng `EnumMap` để lưu bảng chuyển trạng thái hợp lệ.

PHẦN 12. COLLECTIONS FRAMEWORK BÊN TRONG

12.1 Bản đồ cấu trúc dữ liệu và độ phức tạp

Cấu trúc	Bên trong	get/contains	add	remove	Ghi chú
ArrayList	Mảng động	O(1) theo chỉ số, O(n) tìm	O(1) amortized cuối, O(n) giữa	O(n)	Mặc định cho List
LinkedList	Danh sách liên kết đôi	O(n)	O(1) đầu/cuối	O(1) nếu có iterator	Hiếm khi nhanh hơn ArrayList do cache locality kém
ArrayDeque	Mảng vòng	O(1) hai đầu	O(1)	O(1) hai đầu	Thay cho Stack và Queue
HashMap / HashSet	Bảng băm	O(1) trung bình	O(1)	O(1)	Không thứ tự
LinkedHashMap	HashMap + danh sách liên kết đôi	O(1)	O(1)	O(1)	Giữ thứ tự chèn hoặc truy cập
TreeMap / TreeSet	Cây đỏ đen	O(log n)	O(log n)	O(log n)	Có thứ tự, NavigableMap
PriorityQueue	Heap nhị phân trong mảng	O(1) xem đỉnh	O(log n)	O(log n)	Duyệt không theo thứ tự
EnumMap / EnumSet	Mảng / bit vector	O(1)	O(1)	O(1)	Nhanh nhất cho khóa enum
BitSet	Mảng long	O(1)	O(1)	O(1)	Tập số nguyên dày đặc

12.2 Chi tiết từng cấu trúc

ArrayList

- Mảng `Object[] elementData`; khi đầy tăng lên **1.5 lần** (`newCap = old + (old >> 1)`) bằng `Arrays.copyOf`. Nếu biết trước kích thước, dùng `new ArrayList<>(n)` để tránh sao chép.
- `modCount` đếm số lần thay đổi cấu trúc; iterator so sánh `modCount` để ném `ConcurrentModificationException` (cơ chế **fail-fast**, chỉ là best-effort, không đảm bảo trong đa luồng).
- `subList(a, b)` là **view** chia sẻ dữ liệu với list gốc, không phải bản sao: sửa list gốc cấu trúc sau khi tạo subList làm subList dùng được là lỗi. Muốn bản sao dùng `new ArrayList<>(list.subList(a, b))`.
- Xóa nhiều phần tử: dùng `removeIf` (O(n) một lần) thay vì gọi `remove` trong vòng lặp (O(n²)).

Arrays.asList / List.of / unmodifiable

Cách tạo	Đặc tính
`Arrays.asList(arr)`	Kích thước cố định, `set` được, ghi xuyên xuống mảng gốc; `add/remove` ném `UnsupportedOperationException`
`Collections.unmodifiableList(l)`	**View** chỉ đọc; nếu list gốc đổi thì view đổi theo
`List.of(...)`, `List.copyOf(l)`	Bất biến thật sự, **không chấp nhận null** (ném NPE), `copyOf` sao chép

ArrayDeque: mảng vòng với hai chỉ số `head` và `tail` (các bản trước JDK 11 bắt buộc kích thước là lũy thừa 2 để dùng phép AND thay modulo; từ JDK 11 cài đặt đã đổi nhưng ý tưởng mảng vòng vẫn giữ); không cho phép null. Dùng làm ngăn xếp (`push/pop`) và hàng đợi (`offer/poll`); `java.util.Stack` và `Vector` là di sản cũ, đồng bộ không cần thiết.

PriorityQueue: heap nhị phân lưu trong mảng, con của chỉ số `i` ở `2i+1` và `2i+2`. `offer` làm `siftUp`, `poll` lấy đỉnh rồi `siftDown` phần tử cuối. Khởi tạo từ collection có sẵn dùng `heapify` O(n). Duyệt hoặc `toString` **không** ra thứ tự sắp xếp, chỉ `poll` liên tục mới ra đúng thứ tự.

TreeMap: cây đỏ đen tự cân bằng, sắp theo `Comparable` hoặc `Comparator`. Có các thao tác điều hướng: `floorKey`, `ceilingKey`, `higherKey`, `lowerKey`, `headMap`, `tailMap`, `subMap`, `descendingMap`. **Tính tương đương của cây do comparator quyết định, không phải `equals`**: comparator trả 0 nghĩa là trùng khóa.

LinkedHashMap: kế thừa HashMap, thêm danh sách đôi nối các entry. Với `accessOrder = true` mỗi lần `get/put` đưa entry lên cuối, cùng `removeEldestEntry` làm **LRU cache** chỉ vài dòng:

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int max;
    LruCache(int max) { super(16, 0.75f, true); this.max = max; }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > max;
    }
}
```

Các Map đặc biệt: `IdentityHashMap` (so khóa bằng `==`), `WeakHashMap` (khóa là tham chiếu yếu, entry tự biến mất khi khóa bị GC, hợp cho cache metadata), `ConcurrentSkipListMap` (có thứ tự, thread-safe).

12.3 Sắp xếp

- Mảng primitive: **Dual-Pivot Quicksort** (không ổn định, nhưng với primitive không cần ổn định).
- Mảng object và `List.sort` / `Collections.sort`: **TimSort** (kết hợp merge sort và insertion sort, **ổn định**, tận dụng các đoạn đã có thứ tự, O(n) với dữ liệu gần sắp xếp).
- Comparator phải thỏa hợp đồng (phản xứng, bắc cầu, nhất quán). Comparator sai có thể gây `IllegalArgumentException: Comparison method violates its general contract!`. Lỗi kinh điển: dùng phép trừ `a - b` để so sánh int (tràn số); dùng `Integer.compare(a, b)`.
- Dựng comparator bằng `Comparator.comparing(Person::age).thenComparing(Person::name).reversed()`; xử lý null với `nullsFirst/nullsLast`.

12.4 Chọn đúng cấu trúc

- Danh sách tổng quát: `ArrayList`. Hàng đợi hoặc ngăn xếp: `ArrayDeque`. Tra cứu theo khóa: `HashMap`. Cần thứ tự chèn: `LinkedHashMap`. Cần sắp xếp hoặc truy vấn khoảng: `TreeMap`. Top-K phần tử: `PriorityQueue` kích thước K (heap nhỏ nhất giữ K phần tử lớn nhất). Khóa là enum: `EnumMap`.
- Dữ liệu dùng chung nhiều thread: xem mục 5.5 (`ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`).

- Với kiểu primitive số lượng rất lớn, `List<Integer>` tốn bộ nhớ gấp nhiều lần mảng `int[]` (mỗi Integer là một object 16 byte cộng tham chiếu 4 đến 8 byte); cân nhắc `int[]` hoặc thư viện như Eclipse Collections, fastutil.

12.5 Bài tập Phần 12

A. Dự đoán kết quả

1. `List<Integer> l = new ArrayList<>(List.of(1, 2, 3, 4)); for (Integer x : l) if (x == 2) l.remove(x);` có ném exception không? Thử lại với `x == 3` thì sao?
2. `String[] a = {"x", "y"}; List<String> l = Arrays.asList(a); a[0] = "z"; System.out.println(l.get(0)); l.add("w");`
3. `TreeSet<BigDecimal> t = new TreeSet<>(); t.add(new BigDecimal("2.0")); t.add(new BigDecimal("2.00")); HashSet<BigDecimal> h = new HashSet<>(t);` kích thước `t` và `h` lần lượt là bao nhiêu?
4. `PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 4, 2, 3)); System.out.println(pq);` có chắc in `[1, 2, 3, 4, 5]` không?

Đáp án gợi ý: (1) Với `x == 2` **ném** `ConcurrentModificationException`: sau khi xóa, `hasNext()` vẫn true và `next()` phát hiện `modCount` đã đổi. Với `x == 3` (phần tử **kế cuối**) **không** ném exception vì sau khi xóa `cursor == size` nên `hasNext()` trả false và vòng lặp kết thúc sớm; đó là hành vi trùng hợp, không được dựa vào. (2) In `z` (ghi xuyên xuống mảng gốc), sau đó `add` ném `UnsupportedOperationException`. (3) `t` có 1 phần tử (`compareTo` coi hai giá trị bằng nhau). `new HashSet<>(t)` chỉ sao chép những gì `t` đang có nên cũng chỉ có 1 phần tử; còn nếu `add` cả hai giá trị trực tiếp vào một `HashSet` thì có 2 phần tử, vì `equals` phân biệt scale 2.0 và 2.00. (4) Không chắc; heap chỉ đảm bảo phần tử đầu nhỏ nhất, thứ tự còn lại theo cấu trúc cây.

B. Thực nghiệm

5. Đo bằng JMH: thêm 1 triệu phần tử vào `ArrayList` có và không có dung lượng ban đầu; chèn vào đầu `ArrayList` so với `LinkedList` so với `ArrayDeque` với 100.000 phần tử. Giải thích kết quả bằng cache locality.
6. Gây va chạm hash có chủ đích: tạo class có `hashCode` luôn trả 1, đo thời gian `get` khi có 100.000 khóa với `Comparable` và không `Comparable`; quan sát hiệu ứng treeify.
7. Dùng `WeakHashMap`, tạo khóa, bỏ tham chiếu, gọi `System.gc()` và quan sát entry biến mất; so sánh với `HashMap`.

C. Lập trình

8. Cài **LRU cache** (a) bằng `LinkedHashMap`, (b) tự làm bằng `HashMap` và danh sách liên kết đôi, thao tác O(1). Viết test với nhiều thread rồi bổ sung đồng bộ.
9. Cài `MinHeap<T>` bằng mảng với `offer`, `poll`, `siftUp`, `siftDown`, so sánh với `PriorityQueue`.
10. Tìm K phần tử lớn nhất trong luồng 10 triệu số mà chỉ dùng bộ nhớ O(K).

PHẦN 13. STREAM, OPTIONAL VÀ JAVA.TIME

13.1 Stream hoạt động thế nào

Stream không lưu dữ liệu; nó mô tả một **pipeline** gồm nguồn (collection, mảng, generator) → các thao tác trung gian (lazy) → một thao tác kết thúc (eager, kích hoạt toàn bộ). Cơ chế bên trong:

- **Spliterator** là bộ duyệt có khả năng chia nhỏ (`trySplit`) và mang các **characteristics** (`SIZED`, `ORDERED`, `DISTINCT`, `SORTED`, `IMMUTABLE`...). Characteristics giúp tối ưu: ví dụ nguồn `SIZED` biết trước kích thước để cấp phát chính xác.
- Mỗi thao tác trung gian bọc thành một **Sink** nối tiếp nhau; mỗi phần tử đi qua cả chuỗi sink **một lần** (loop fusion), không tạo collection trung gian. Vì vậy `filter` rồi `map` rồi `limit(3)` có thể chỉ xử lý vài phần tử đầu của nguồn vô hạn.
- Thao tác **stateful** (`sorted`, `distinct`, `skip`/`limit` trên stream có thứ tự) có thể buộc phải giữ lại dữ liệu (rào chắn), làm giảm lợi thế lazy và song song.
- Thứ tự thực thi quan sát được: với `peek` hoặc `forEach` in log, các phần tử đi **hết pipeline từng cái một**, không phải từng bước cho toàn bộ danh sách.

```
Stream.of("a", "bb", "ccc")
      .peek(s -> System.out.println("peek " + s))
      .filter(s -> s.length() > 1)
      .map(String::toUpperCase)
      .findFirst();            // chỉ in peek a, peek bb rồi dừng
```

13.2 Collector

Một `Collector` gồm bốn hàm: `supplier` (tạo hộp chứa), `accumulator` (thêm phần tử), `combiner` (gộp hai hộp, dùng khi song song), `finisher` (biến đổi cuối).

```
// Thống kê tổng hợp
var stats = list.stream().collect(Collectors.summarizingInt(Order::amount));
// Hai kết quả cùng lúc (Java 12+)
var r = list.stream().collect(Collectors.teeing(
    Collectors.counting(), Collectors.averagingInt(Order::amount), (n, avg) -
> n + ":" + avg));
// Nhóm nhiều tầng
Map<String, Map<Boolean, List<Order>>> m = orders.stream().collect(
    Collectors.groupingBy(Order::city, Collectors.partitioningBy(o ->
o.amount() > 100)));
// Collector tự viết
Collector<String, StringJoiner, String> joiner = Collector.of(
    () -> new StringJoiner(", "), StringJoiner::add, StringJoiner::merge,
StringJoiner::toString);
```

Lưu ý: `Collectors.toMap` ném `IllegalStateException` khi trùng khóa trừ khi truyền hàm gộp; `toList()` trả list **có thể thay đổi** nhưng không đảm bảo kiểu, còn `Stream.toList()` (Java 16) trả list **không thể sửa**; `groupingBy` không đảm bảo kiểu Map (truyền `TreeMap::new` nếu cần săp xếp).

13.3 Stream song song

`parallelStream()` chia nguồn băng `trySplit` rồi chạy các phần trên `ForkJoinPool.commonPool()`. Chỉ có lợi khi: dữ liệu lớn; thao tác **nặng CPU và độc lập**; nguồn chia dễ (`ArrayList`, mảng, `IntStream.range`; kém: `LinkedList`, `Stream.iterate`); không có side effect, không chặn I/O (sẽ làm cạn commonPool dùng chung với cả ứng dụng). Hàm `reduce` phải **kết hợp (associative)** và `identity` đúng thì kết quả mới đúng khi song song. Thao tác `forEach` trong parallel không bảo đảm thứ tự (dùng `forEachOrdered` nếu cần, nhưng mất lợi thế). Luôn **đo trước khi dùng**.

13.4 Optional dùng đúng

- Dùng làm **kiểu trả về** để thể hiện "có thể không có kết quả". Không dùng cho field, tham số, hay phần tử collection; không dùng `Optional.get()` trần mà không kiểm tra.
- Ưu tiên: `map`, `flatMap`, `filter`, `orElse`, `orElseGet` (tính lười, dùng khi giá trị mặc định tốn kém), `orElseThrow`, `ifPresentOrElse`.
- `orElse(expensive())` luôn gọi `expensive()` kể cả khi có giá trị; `orElseGet(() -> expensive())` chỉ gọi khi cần.
- Với số có kiểu primitive dùng `OptionalInt/Long/Double`.

13.5 java.time

Tất cả class **bất biến và thread-safe** (khác `Date`, `Calendar`, `SimpleDateFormat`).

Class	Ý nghĩa
`Instant`	Một thời điểm trên trục thời gian (UTC), độ chính xác nano giây
`LocalDate`, `LocalTime`, `LocalDateTime`	Ngày/giờ **không** gắn múi giờ (dạng "lịch treo tường")
`ZonedDateTime`, `OffsetDateTime`	Thời điểm kèm múi giờ (có quy tắc DST) hoặc chỉ kèm độ lệch UTC
`Duration`	Khoảng thời gian dựa trên giây/nano (máy tính)
`Period`	Khoảng thời gian theo ngày/tháng/năm (lịch)
`DateTimeFormatter`	Định dạng/phân tích, thread-safe

Nguyên tắc: **lưu và truyền thời điểm bằng `Instant` (hoặc UTC), chỉ chuyển sang múi giờ khi hiển thị**. Cạm bẫy: DST làm một ngày có 23 hoặc 25 giờ nên `plus(Duration.ofDays(1))` và `plusDays(1)` trên `ZonedDateTime` có thể khác nhau; `LocalDateTime` không đủ thông tin để so sánh hai thời điểm giữa các múi giờ; kiểm thử nên **tiêm `Clock`** (`Instant.now(clock)`) thay vì gọi `now()` trực tiếp để có thể cố định thời gian trong test.

13.6 Bài tập Phần 13

A. Dự đoán kết quả

1. `Stream.of(1, 2, 3, 4).filter(x -> { System.out.println("f" + x); return x % 2 == 0; }).map(x -> { System.out.println("m" + x); return x; }).findFirst();` in ra những dòng nào, theo thứ tự nào?
2. `Stream<Integer> s = Stream.of(1, 2); s.forEach(x -> {}); s.forEach(x -> {});` điều gì xảy ra?
3. `Optional.of(1).orElse(compute())` có gọi `compute()` không? Với `orElseGet` thì sao?
4. `Collectors.toMap(String::length, s -> s)` trên `["a", "b"]` cho kết quả gì?

5. `ZonedDateTime` vào đêm chuyển sang giờ mùa hè: `plusDays(1)` và `plus(Duration.ofHours(24))` khác nhau thế nào?

Đáp án gợi ý: (1) `f1`, `f2`, `m2` rồi dừng (lazy, short-circuit, từng phần tử). (2) `IllegalStateException: stream has already been operated upon or closed`. (3) Có (`orElse` tính ngay); `orElseGet` thì không. (4) `IllegalStateException: Duplicate key 1`. (5) `plusDays(1)` giữ nguyên giờ trên đồng hồ (ngày đó có 23 giờ thực), `plus(24h)` cộng đủ 24 giờ nên đồng hồ lệch 1 giờ.

B. Thực nghiệm

6. Dùng JMH so sánh tính tổng 10 triệu số bằng vòng for, `stream()`, `parallelStream()`, `IntStream.range().parallel()`. Tìm ngưỡng dữ liệu để parallel bắt đầu có lợi, và quan sát khi thao tác bên trong có `Thread.sleep` hoặc I/O.
7. Tạo `Stream.iterate` vô hạn rồi dùng `limit` / `takeWhile`; thử đặt `sorted()` trước `limit` và giải thích vì sao treo.
8. Thử gọi `parallelStream` với hàm `reduce` không kết hợp (ví dụ phép trừ) và quan sát kết quả sai, không ổn định.

C. Lập trình

9. Viết `Collector` tự làm tính trung vị (median) của luồng số.
10. Cho danh sách đơn hàng, dùng một pipeline duy nhất tính: doanh thu theo thành phố, top 3 khách hàng chi nhiều nhất, tỉ lệ đơn lớn hơn 100 theo tháng.
11. Viết hàm tính số ngày làm việc giữa hai `LocalDate` (bỏ cuối tuần và danh sách ngày lễ), có test cố định thời gian bằng `Clock`.

PHẦN 14. I/O, NIO VÀ MẠNG

14.1 I/O cổ điển và mô hình decorator

- Hai họ: **byte stream** (`InputStream/OutputStream`) và **character stream** (`Reader/Writer`, luôn gắn với một charset). `InputStreamReader` là cầu nối byte → char.
- Các lớp được **bọc lồng** (decorator): `new BufferedReader(new InputStreamReader(new FileInputStream(f), UTF_8))`. Bộ đệm giảm số lần gọi hệ thống (syscall), vì mỗi lệnh đọc/ghi không đệm xuống thẳng hệ điều hành. Đọc từng byte từ `FileInputStream` không đệm có thể chậm hơn hàng chục lần.

- Ghi qua bộ đệm cần `flush()` hoặc `close()` (close tự flush). Chỉ cần đóng lớp bọc ngoài cùng, các lớp bên trong đóng theo; dùng try-with-resources.
- Java hiện đại ưu tiên `Files.readString`, `Files.readAllLines`, `Files.newBufferedReader(path, UTF_8)`, `Files.lines` (Stream **phải được đóng**).

14.2 NIO: Buffer và Channel

Buffer có bốn con số: `capacity` (tối đa), `position` (vị trí đọc/ghi hiện tại), `limit` (giới hạn được phép), `mark`. Luồng dùng điển hình:

```
ByteBuffer buf = ByteBuffer.allocate(8192);
while (channel.read(buf) != -1) {   // channel ghi vào buffer (chế độ ghi)
    buf.flip();                     // đổi sang chế độ đọc: limit = position,
position = 0
    while (buf.hasRemaining()) out.write(buf);
    buf.clear();                    // quay lại chế độ ghi (hoặc compact() để
giữ phần chưa đọc)
}
```

Thao tác	Tác dụng
`flip()`	Từ ghi sang đọc
`clear()`	Reset để ghi lại từ đầu (dữ liệu chưa bị xóa thật)
`compact()`	Dồn phần chưa đọc lên đầu rồi tiếp tục ghi
`rewind()`	Đọc lại từ đầu
`slice()`, `duplicate()`	View chia sẻ dữ liệu, con trỏ độc lập

- **Heap buffer** (`allocate`) nằm trong heap Java, dễ GC nhưng khi I/O phải sao chép sang bộ nhớ native. **Direct buffer** (`allocateDirect`) nằm ngoài heap, I/O không cần sao chép trung gian nhưng cấp phát/ giải phóng đắt và chịu `-XX:MaxDirectMemorySize`; nên tái sử dụng (pool).
- **Zero-copy**: `FileChannel.transferTo/transferFrom` nhờ hệ điều hành chuyển dữ liệu giữa file và socket mà không đi qua user space (cơ sở của hiệu năng web server phục vụ file tĩnh).
- **MappedByteBuffer** (`FileChannel.map`) ánh xạ file vào bộ nhớ ảo (mmap), truy cập như mảng; rất tốt cho file lớn truy cập ngẫu nhiên, nhưng giải phóng bộ nhớ không tường minh (phụ thuộc GC) và lỗi I/O có thể bùng ra bất ngờ.

- Nhớ đặt `ByteOrder` khi trao đổi nhị phân với hệ thống khác (mặc định Java là big-endian).

14.3 Selector và mô hình Reactor

Vấn đề **C10K**: một thread cho mỗi kết nối không mở rộng được tới hàng chục nghìn kết nối (tốn bộ nhớ stack và context switch). Giải pháp NIO: **một thread (hoặc ít thread) giám sát nhiều channel** thông qua `Selector` (bên dưới là epoll/kqueue):

```
Selector selector = Selector.open();
ServerSocketChannel server = ServerSocketChannel.open();
server.bind(new InetSocketAddress(8080));
server.configureBlocking(false);
server.register(selector, SelectionKey.OP_ACCEPT);

while (true) {
    selector.select();                                  // chặn đến khi có sự
kiện
    var it = selector.selectedKeys().iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next(); it.remove();      // phải remove, nếu
không xử lý lặp
        if (key.isAcceptable()) {
            SocketChannel c = server.accept();
            c.configureBlocking(false);
            c.register(selector, SelectionKey.OP_READ,
ByteBuffer.allocate(1024));
        } else if (key.isReadable()) {
            SocketChannel c = (SocketChannel) key.channel();
            ByteBuffer b = (ByteBuffer) key.attachment();
            if (c.read(b) == -1) { c.close(); continue; }
            b.flip(); c.write(b); b.compact();          // echo
        }
    }
}
```

Các khung như **Netty** xây thêm: nhóm thread boss/worker (event loop), pipeline xử lý, quản lý buffer (pooled), xử lý ghép/tách gói (framing) mà tự viết NIO thô rất dễ sai (đọc nửa gói, ghi dở).

Chọn mô hình

Mô hình	Ưu	Nhược
Thread mỗi kết nối (blocking)	Code tuần tự, dễ đọc	Tốn tài nguyên nếu dùng platform thread
Reactor (Selector, Netty)	Mở rộng rất tốt	Code bất đồng bộ phức tạp, tuyệt đối không được chặn event loop
Thread mỗi kết nối + **virtual thread** (Java 21)	Code tuần tự mà vẫn mở rộng lớn	Không giúp việc nặng CPU, cần chú ý pinning

14.4 NIO.2: Path và Files

- `Path` thay `File`: `Path.of("a", "b.txt")`, `resolve`, `relativize`, `normalize`, `toAbsolutePath`.
- `Files.walk/find/list` (trả Stream, phải đóng), `walkFileTree` với `FileVisitor` để xử lý cây thư mục, `Files.copy/move` (với `ATOMIC_MOVE`, `REPLACE_EXISTING`), `Files.createTempFile`, đọc thuộc tính (`BasicFileAttributes`, `PosixFilePermissions`), symbolic link.
- **Ghi file an toàn**: ghi vào file tạm cùng thư mục rồi `Files.move(tmp, target, ATOMIC_MOVE)` để người đọc không bao giờ thấy file ghi dở.
- `WatchService` theo dõi thay đổi thư mục (dựa trên inotify trên Linux); sự kiện có thể gộp hoặc trùng nên cần xử lý idempotent.

14.5 Mạng

- **TCP**: bắt tay ba bước; kết nối đóng để lại trạng thái `TIME_WAIT` (hàng chục nghìn kết nối ngắn có thể cạn cổng, nên dùng connection pool và keep-alive). `ServerSocket.bind` với `backlog` quyết định hàng đợi kết nối chờ accept. `SO_REUSEADDR` giúp khởi động lại server nhanh. `TCP_NODELAY` tắt thuật toán Nagle để giảm độ trễ cho gói nhỏ.
- **Timeout là bắt buộc**: `connectTimeout` (chờ thiết lập kết nối) và `SO_TIMEOUT`/read timeout (chờ dữ liệu) khác nhau; thiếu timeout là nguyên nhân phổ biến của thread bị treo vô hạn (thấy ở `socketRead0` trong thread dump).
- **HTTP Client chuẩn** (Java 11+):

```
HttpClient client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_2)
    .connectTimeout(Duration.ofSeconds(3)).build();
HttpRequest req = HttpRequest.newBuilder(URI.create("https://example.com"))
    .timeout(Duration.ofSeconds(5)).GET().build();
HttpResponse<String> res = client.send(req,
HttpResponse.BodyHandlers.ofString());
client.sendAsync(req, HttpResponse.BodyHandlers.ofString())      // bất đồng
bộ (CompletableFuture)
      .thenApply(HttpResponse::body);
```

Tái sử dụng một `HttpClient` (nó quản lý pool kết nối). `URI` chỉ phân tích cú pháp; `URL.equals/hashCode` có thể **thực hiện tra cứu DNS** (nên dùng `URI` cho so sánh/Map key). JVM cache kết quả DNS (`networkaddress.cache.ttl`), cần lưu ý khi IP đổi.

- **TLS**: `SSLContext` , trust store (`cacerts`) và key store. Không tắt kiểm tra chứng chỉ trong môi trường thật. Dùng `-Djavax.net.debug=ssl:handshake` để gỡ lỗi bắt tay.

14.6 Bài tập Phần 14

A. Dự đoán và giải thích

1. Vì sao sau `buf.put(data)` mà quên `flip()` rồi `channel.write(buf)` thì ghi ra 0 byte?
2. Hai cách đọc file 1GB theo từng byte: `FileInputStream` thẳng và bọc `BufferedInputStream` . Cách nào nhanh hơn, ước lượng chênh lệch bao nhiêu lần và vì sao?
3. Server NIO không gọi `it.remove()` sau mỗi key thì hiện tượng gì xảy ra?

Đáp án gợi ý: (1) Sau `put` , `position` đang ở cuối dữ liệu và `limit = capacity` , nên phần "còn lại để đọc" (`limit - position`) là phần trống phía sau, không phải dữ liệu vừa ghi; cần `flip()` . (2) Có bộ đệm nhanh hơn rất nhiều (thường hàng chục lần) vì giảm số syscall từ một cho mỗi byte xuống một cho mỗi khối vài KB. (3) Key đã xử lý vẫn nằm trong `selectedKeys` và bị xử lý lại ở vòng sau (gây lỗi hoặc vòng lặp bận).

B. Thực nghiệm

4. Chép file 2GB bằng bốn cách: stream có đệm, `Files.copy` , `FileChannel.transferTo` , và đọc/ghi qua direct buffer. Đo thời gian và dùng `strace -c` (Linux) hoặc công cụ tương đương để so sánh số syscall.

5. Viết server echo ba phiên bản: (a) thread pool cố định, (b) Selector đơn thread, (c) virtual thread mỗi kết nối. Dùng công cụ tải (wrk, hey, hoặc tự viết client) mở 10.000 kết nối đồng thời; so sánh bộ nhớ (RSS), số thread (`jcmd Thread.print`), độ trễ p99.
6. Dùng `FileChannel.map` đọc ngẫu nhiên file 4GB và so sánh với `RandomAccessFile`.
7. Bắt buộc timeout: viết client kết nối tới cổng không phản hồi (không có timeout, rồi có timeout), dùng `jstack` quan sát thread bị kẹt ở đâu.

C. Lập trình

8. Viết bộ đọc khung (framing) cho giao thức dạng `[4 byte độ dài][nội dung]` trên NIO không chặn, xử lý được gói bị tách nhỏ và nhiều gói dính liền.
9. Viết công cụ `dirsize` dùng `Files.walkFileTree` tính dung lượng thư mục, bỏ qua symbolic link, xử lý được thư mục không có quyền đọc.
10. Viết hàm ghi file cấu hình an toàn (ghi tạm rồi `ATOMIC_MOVE`) và chứng minh bằng cách ngắt tiến trình giữa chừng rằng file đích không bao giờ ở trạng thái dở dang.

PHẦN 15. RUNTIME NÂNG CAO VÀ TRIỂN KHAI

Lưu ý về tính cập nhật: các API ở phần này thay đổi nhanh qua từng bản Java (preview, deprecated, final). Trước khi dùng, **đối chiếu JEP và Javadoc của bản JDK bạn đang chạy**.

15.1 MethodHandle và VarHandle

- **MethodHandle** (`java.lang.invoke`) là tham chiếu có kiểu tới method, field hoặc constructor, tìm qua `MethodHandles.lookup()`. Khi nằm trong biến `static final`, JIT coi như hằng và có thể inline, nên nhanh gần bằng gọi trực tiếp (reflection thì không). `invokeExact` yêu cầu đúng kiểu từng tham số, `invoke` tự thích ứng kiểu.
- **VarHandle** là tham chiếu có kiểu tới biến (field, phần tử mảng) với các **chế độ truy cập** từ yếu tới mạnh:

Chế độ	Đảm bảo
plain (`get/set`)	Như biến thường, không đảm bảo hiển thị
opaque (`getOpaque/setOpaque`)	Đảm bảo tiến triển và nguyên tử từng biến, không đảm bảo thứ tự giữa các biến
acquire/release (`getAcquire/setRelease`)	Thứ tự một chiều, đủ cho nhiều mẫu publish-consume, rẻ hơn volatile

Chế độ	Đảm bảo
volatile (`getVolatile/setVolatile`)	Tương đương `volatile`
Nguyên tử	`compareAndSet` , `getAndAdd` , `getAndSet` …

VarHandle là nền tảng thay thế `Unsafe` cho cấu trúc lock-free (các lớp `Atomic*Field*` , `ConcurrentLinkedQueue` … đều dùng nó). Dành cho tác giả thư viện; ứng dụng thông thường hãy dùng lớp `java.util.concurrent` có sẵn.

15.2 Unsafe, FFM API và JNI

- `sun.misc.Unsafe` (thao tác bộ nhớ thô, CAS, park/unpark) được nhiều thư viện dùng (Netty, Kryo, Hazelcast) nhưng **không phải API chuẩn** và đang bị loại dần (các JEP gần đây đã đánh dấu deprecated for removal các phương thức truy cập bộ nhớ). Thay thế: **VarHandle** cho truy cập biến, **FFM API** cho bộ nhớ ngoài heap và gọi mã native.
- **FFM (Foreign Function & Memory) API** (Project Panama): xem trước (preview) ở Java 21, **chính thức từ Java 22**. Thành phần: `Arena` (vòng đời bộ nhớ, có kiểm tra giới hạn và thời gian sống, tránh use-after-free), `MemorySegment` , `Linker` + `FunctionDescriptor` + `MethodHandle` để gọi hàm C:

```
Linker linker = Linker.nativeLinker();
MethodHandle strlen = linker.downcallHandle(
    linker.defaultLookup().find("strlen").orElseThrow(),
    FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));
try (Arena arena = Arena.ofConfined()) {
    MemorySegment s = arena.allocateFrom("hello");   // Java 21 (preview):
allocateUtf8String
    long n = (long) strlen.invokeExact(s);            // 5
}
```

- **JNI** (Java Native Interface): khai báo `native` method, sinh header bằng `javac -h` , nạp thư viện bằng `System.loadLibrary` . Rủi ro: lỗi con trỏ làm **sập cả JVM**, quản lý tham chiếu local/global dễ rò rỉ, chi phí chuyển đổi và khó debug. Dự án mới nên ưu tiên FFM.

15.3 Java agent và thao tác bytecode

- Agent là JAR có `premain(String, Instrumentation)` (nạp lúc khởi động qua `-javaagent:agent.jar`) hoặc `agentmain` (gắn vào JVM đang chạy). `Instrumentation.addTransformer(ClassFileTransformer)` cho phép **sửa bytecode**

của class ngay khi được nạp.

- Dùng cho: APM/tracing (Datadog, OpenTelemetry), profiler, code coverage (JaCoCo), mock "inline" (Mockito), hot reload.
- Công cụ sửa bytecode: **ASM** (cấp thấp, nhanh), **ByteBuddy** (API thân thiện, dùng bởi Mockito, Hibernate), **Javassist**. Java đang chuẩn hóa **Class-File API** (xem JEP hiện hành).
- Từ Java 21, việc tự động gắn agent động vào JVM đang chạy bị cảnh báo và sẽ bị hạn chế hơn; nên nạp agent bằng `-javaagent` ngay từ đầu.

15.4 ServiceLoader và SPI

Cơ chế **plug-in**: giao diện (interface) khai báo ở phía lõi, nhà cung cấp đặt file `META-INF/services/<tên interface>` (hoặc `provides ... with ...` trong `module-info`), lõi nạp bằng `ServiceLoader.load(Interface.class)`. JDBC driver, logging (SLF4J), `java.nio.file.spi` đều dùng cách này. Duyệt `ServiceLoader` là **lazy**; mỗi lần `load` là một bộ nạp mới.

15.5 Hệ thống module (JPMS)

```
module com.shop.order {
    requires java.sql;
    requires transitive com.shop.core;       // người dùng order cũng thấy
core
    exports com.shop.order.api;              // công khai package này
    exports com.shop.order.internal to com.shop.test;   // chỉ cho module cụ
thể
    opens com.shop.order.model to com.fasterxml.jackson.databind;  // mở cho
reflection
    uses com.shop.core.PaymentProvider;
    provides com.shop.core.PaymentProvider with com.shop.order.CardPayment;
}
```

- **Đóng gói mạnh (strong encapsulation)**: package không `exports` thì kể cả `public` cũng không truy cập được từ module khác; package không `opens` thì `setAccessible(true)` bị chặn. Từ Java 17 các package nội bộ của JDK (như `sun.misc`, `jdk.internal.*`) bị niêm phong; vượt tạm bằng `--add-opens` / `--add-exports` (chỉ nên coi là giải pháp tạm khi nâng cấp thư viện cũ).

- **Unnamed module**: mã trên classpath thuộc module này, thấy được mọi thứ. **Automatic module**: JAR thường đặt trên module path, tên suy ra từ tên file, xuất mọi thứ. Không được có **split package** (cùng package ở hai module).
- Công cụ: `jdeps` (phân tích phụ thuộc), `jlink` (tạo runtime thu gọn chỉ gồm module cần), `jpackage` (đóng gói bộ cài). Spring, Hibernate, Jackson phần lớn vẫn chạy trên classpath; nhiều dự án dùng module chủ yếu nhờ `jlink`.

15.6 Khởi động nhanh và triển khai

Kỹ thuật	Ý tưởng	Đánh đổi
AppCDS (Class Data Sharing)	Lưu sẵn class đã parse/verify vào archive, ánh xạ vào bộ nhớ	Giảm thời gian khởi động và bộ nhớ, cần bước tạo archive
`jlink`	Runtime chỉ chứa module cần dùng	Ảnh nhỏ gọn; cần module hóa hoặc dùng `jdeps` xác định
GraalVM Native Image	Biên dịch trước (AOT) thành file nhị phân	Khởi động tính bằng ms, bộ nhớ thấp; **closed world**: reflection, proxy, resource cần khai báo cấu hình; thường đạt đỉnh hiệu năng thấp hơn JIT
CRaC / checkpoint	Chụp snapshot JVM đã warm-up rồi khôi phục	Khởi động rất nhanh; phụ thuộc hỗ trợ của JDK và nền tảng

Trong container, nhớ cho JVM biết giới hạn tài nguyên (`-XX:MaxRAMPercentage` , `-XX:ActiveProcessorCount` khi cần) và để ý số GC thread, số thread pool mặc định tính theo CPU nhìn thấy.

15.7 Các dự án lớn của OpenJDK (theo hiểu biết đến giữa năm 2026)

Mỗi dự án phát hành dần qua các JEP; **hãy kiểm tra trạng thái hiện tại** tại openjdk.org/jeps.

Dự án	Nội dung	Tình trạng ghi nhận
Loom	Virtual threads, structured concurrency, scoped values	Virtual threads chính thức ở Java 21; các phần còn lại phát triển qua preview

Dự án	Nội dung	Tình trạng ghi nhận
Panama	FFM API, vector API	FFM chính thức ở Java 22; Vector API vẫn incubator
Amber	Record, sealed, pattern matching, text block...	Nhiều tính năng đã chính thức, tiếp tục bổ sung
Valhalla	Value class (đối tượng không có danh tính), cải thiện bố cục bộ nhớ	Đang phát triển, cần theo dõi
Leyden	Giảm thời gian khởi động và warm-up (AOT cache)	Đang phát triển, một số phần đã vào JDK

15.8 Bài tập Phần 15

A. Dự đoán và giải thích

1. Vì sao `MethodHandle` lưu trong biến `static final` thường nhanh hơn khi lưu trong biến instance thường?
2. Một thư viện cũ gọi `field.setAccessible(true)` vào class `java.base` và ném `InaccessibleObjectException` trên Java 17. Hai cách xử lý khác nhau là gì và cách nào nên ưu tiên?
3. Hai module cùng chứa package `com.util` thì điều gì xảy ra khi khởi động?

Đáp án gợi ý: (1) JIT coi `static final` là hằng nên có thể inline mục tiêu của handle; biến thường thì handle là giá trị chưa biết lúc compile nên phải gọi gián tiếp. (2) Thêm `--add-opens java.base/<package>=ALL-UNNAMED` (tạm thời) hoặc nâng cấp/thay thư viện dùng API chính thức; ưu tiên nâng cấp. (3) Lỗi: split package không được phép trong cùng một tầng module (JVM từ chối khởi động hoặc nạp module).

B. Thực nghiệm

6. JMH so sánh bốn cách gọi cùng một method: gọi trực tiếp, reflection `Method.invoke`, `MethodHandle` trong `static final`, `MethodHandle` trong field thường.
7. Viết một Java agent đơn giản in ra tên mỗi class được nạp (`ClassFileTransformer`), chạy với `-javaagent`. Mở rộng: dùng ByteBuddy hoặc ASM chèn log vào đầu một method.
8. Dùng FFM API (Java 22+) gọi hàm `qsort` hoặc `getpid` của C; so sánh mã với phiên bản JNI tương đương về độ dài và độ an toàn.

9. Tạo ứng dụng nhỏ ba module (`core` , `plugin` , `app`) dùng `ServiceLoader` (`uses/provides`), rồi dùng `jlink` tạo runtime thu gọn; so sánh kích thước với JDK đầy đủ.
10. Cùng một ứng dụng web nhỏ: đo thời gian khởi động và bộ nhớ khi chạy bình thường, khi dùng AppCDS, và (nếu khung hỗ trợ) khi biên dịch Native Image. Lập bảng so sánh.

C. Lập trình

11. Cài bộ đếm không khóa bằng `VarHandle.getAndAdd` trên field `volatile long` , so sánh tốc độ với `AtomicLong` và `LongAdder` dưới tải nhiều thread.
12. Cài **SPSC queue** (một producer một consumer, không khóa) bằng mảng vòng và `VarHandle` với chế độ release/acquire; chứng minh đúng bằng test nhiều lượt.

PHẦN 16. BÀI TẬP CHO PHẦN 1 ĐẾN 9 VÀ DỰ ÁN TỔNG HỢP

Mỗi bài gồm **yêu cầu**, **điều cần quan sát** và (khi phù hợp) **câu hỏi tự kiểm tra**. Công cụ dùng xuyên suốt: `javap` , `jcmd` , `jstat` , `jstack` , `jmap` , JFR/JMC, Eclipse MAT, JMH, JOL, và `jcstress` (kiểm thử đúng đắn của mã đa luồng, phát hiện các kết quả "không thể xảy ra" do reordering). Nên chạy trên máy thật nhiều lõi, tắt chế độ tiết kiệm điện, và **ghi lại số đo kèm cấu hình** (JDK, cờ JVM, CPU) để so sánh.

16.1 JVM và ClassLoader (Phần 1)

1. **Đọc bytecode.** Viết class gồm: hằng `static final` , vòng for, `switch` trên String, lambda, try-with-resources. Chạy `javap -c -p -v` . *Cần quan sát*: lệnh `invoke*` nào được dùng ở đâu; `switch` String sinh ra `hashCode` + `lookupswitch` ; lambda thành `invokedynamic` ; constant pool lưu gì. *Tự kiểm tra*: giải thích vì sao `try-with-resources` sinh ra hai nhánh xử lý exception.
2. **Kích hoạt khởi tạo class.** Cho `class A { static final int K = 1; static final int L = Integer.parseInt("2"); static { System.out.println("A init"); } }` . Dự đoán rồi kiểm chứng khi lần lượt truy cập `A.K` , `A.L` , `new A[1]` , `A.class` , `Class.forName("A")` . *Đáp án gợi ý*: chỉ `A.L` và `Class.forName` làm in ra `A init` (K là hằng compile-time bị inline; `new A[1]` và `A.class` không khởi tạo class).
3. **Hai ClassLoader, hai kiểu.** Viết `MyLoader` nạp class từ một thư mục. Nạp cùng class `Plugin` từ hai instance loader; chứng minh `a.getClass() != b.getClass()` và ép kiểu gây `ClassCastException` dù tên giống nhau.

4. **Hot reload.** Dùng loader mới mỗi lần để nạp lại class `Plugin` sau khi bạn sửa file .class đang chạy; chứng minh class cũ vẫn còn trong bộ nhớ khi còn tham chiếu (và có thể gây rò rỉ Metaspace nếu giữ loader).
5. **Phá delegation.** Viết loader theo kiểu *child-first* cho class của ứng dụng; thử nạp `java.lang.String` bằng nó và quan sát `SecurityException: Prohibited package name`. Vì sao JVM chặn?
6. **Hai lỗi nhìn giống nhau.** Tái hiện riêng rẽ `ClassNotFoundException` (qua `Class.forName`) và `NoClassDefFoundError` (xóa file .class sau khi compile; và trường hợp static init ném exception ở lần dùng đầu tiên).
7. **Độ sâu stack.** Đo độ sâu đệ quy tối đa với `-Xss` khác nhau (256k, 512k, 1m, 4m), và với method có nhiều biến cục bộ hơn. Rút ra mối quan hệ giữa kích thước frame và độ sâu.

16.2 Bộ nhớ và GC (Phần 1.3, 2)

1. **Quan sát các thế hệ.** Chạy chương trình cấp phát liên tục đối tượng sống ngắn và một phần sống dài, với `-XX:+UseParallelGC -Xmn64m -Xmx256m`; theo dõi `jstat -gcutil PID 500`. *Cần quan sát*: Eden đầy → Minor GC; S0/S1 hoán đổi; object được promote; Old tăng dần rồi Full GC. Thay `-XX:MaxTenuringThreshold` (1, 15) và ghi nhận tốc độ promote.
2. **So sánh collector.** Cùng một chương trình cấp phát nặng (hoặc một ứng dụng web nhỏ có tải), chạy với Serial, Parallel, G1, ZGC (cùng `-Xmx`). Điền bảng: thông lượng, pause trung bình và lớn nhất (đọc từ GC log), CPU, bộ nhớ RSS. *Tự kiểm tra*: giải thích bảng bằng đặc điểm thiết kế từng collector.
3. **Các loại tham chiếu.** (a) `SoftReference` làm cache ảnh/ chuỗi lớn: với `-Xmx` nhỏ quan sát khi nào bị xóa; (b) `WeakReference` và `WeakHashMap`: gán null rồi `System.gc()`; (c) `PhantomReference` + `ReferenceQueue` hoặc `Cleaner` để dọn tài nguyên native. *Câu hỏi*: vì sao `System.gc()` chỉ là gợi ý và không nên dùng trong mã thật?
4. **Săn rò rỉ bộ nhớ.** Viết ứng dụng cố ý có ba lỗi: `static Map` không dọn, listener không hủy đăng ký, `ThreadLocal` không `remove` trong thread pool. Chạy với `-XX:+HeapDumpOnOutOfMemoryError -Xmx128m`, mở dump bằng MAT, dùng Dominator Tree và Path to GC Roots để chỉ ra từng lỗi. Viết ghi chú chẩn đoán 5 dòng cho mỗi lỗi.
5. **Humongous allocation (G1).** Cấp phát mảng gần bằng một nửa kích thước region; đọc GC log (`Humongous`), thử `-XX:G1HeapRegionSize` lớn hơn và quan sát khác biệt.
6. **Escape analysis.** JMH hai phiên bản: tạo object nhỏ trong vòng lặp nhưng không thoát khỏi method, chạy với và không với `-XX:-DoEscapeAnalysis`. Dùng `-prof gc` xem `alloc/op`. Giải thích kết quả.

7. **Dự đoán.** `Object a = new Object(); WeakReference<Object> w = new WeakReference<>(a); a = null; System.gc(); System.out.println(w.get());` thường in gì? Nếu còn một biến strong khác giữ object thì sao? *Đáp án gợi ý*: thường là `null` (GC chỉ là gợi ý nhưng phần lớn JVM thực hiện); nếu còn tham chiếu mạnh thì vẫn khác null.

16.3 JIT và công cụ tuning (Phần 2.5, 3)

1. **Warm-up.** Đo thời gian thực hiện 1000 lần gọi một method tính toán, lặp 50 vòng và in từng vòng; chạy với `-Xint`, mặc định, `-XX:TieredStopAtLevel=1`, thêm `-XX:+PrintCompilation`. Vẽ biểu đồ thời gian theo vòng và nhận diện các mức biên dịch.
2. **Inlining.** Dùng `-XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining`. Viết method gọi một method nhỏ (inline được), rồi một method có bytecode lớn hơn 325 byte (`FreqInlineSize` mặc định). Tìm dòng giải thích lý do inline thành công hay thất bại (`too big`, `hot method too big`...).
3. **Call site đa hình.** JMH gọi `interface.method()` với 1, 2, và 3 trở lên cài đặt khác nhau (mono/bi/megamorphic). Đo và giải thích sự khác biệt (profile loại, inline cache, deoptimization).
4. **JFR.** Ghi một chương trình có điểm nóng CPU và cấp phát nhiều (`-XX:StartFlightRecording=duration=60s,filename=a.jfr`), mở bằng JDK Mission Control, chỉ ra method nóng nhất, lớp cấp phát nhiều nhất, và các GC pause.
5. **Diễn tập sự cố.** Viết một ứng dụng cố ý có: một thread quay vòng ăn CPU, hai thread deadlock, một danh sách rò rỉ. Chẩn đoán bằng `top -H -p`, đổi tid sang hex khớp `nid` trong `jstack`, `jcmd PID Thread.print`, `jmap -histo`, và ghi nhật ký chẩn đoán (triệu chứng, bằng chứng, nguyên nhân, cách sửa).

16.4 Đa luồng (Phần 4)

1. **Race condition và bốn cách sửa.** 8 thread cùng tăng một biến 1.000.000 lần. Làm bản `int` thường (kết quả sai), rồi sửa bằng `synchronized`, `ReentrantLock`, `AtomicInteger`, `LongAdder`. Lập bảng thời gian (JMH) và giải thích vì sao khác nhau.
2. **Lỗi hiển thị.** Viết `while (!flag) {}` ở một thread và `flag = true` ở thread khác, `flag` không volatile. Chạy với JVM mặc định và quan sát vòng lặp không thoát (hoặc thử `-Xint` để so sánh). Sửa bằng `volatile`; giải thích vì sao JIT có thể kéo biến ra khỏi vòng lặp.

3. **Reordering với jcstress.** Cài thử nghiệm store-buffering (`x = 1; r1 = y` và `y = 1; r2 = x`); quan sát kết quả `(0, 0)` khi không dùng volatile và biến mất khi dùng volatile. *Câu hỏi*: loại memory barrier nào ngăn kết quả này?
4. **wait/notify.** Cài producer-consumer bằng `wait/notify` có hàng đợi giới hạn. Tạo hai phiên bản lỗi cố ý: dùng `if` thay `while`, và dùng `notify` thay `notifyAll` với nhiều consumer/producer; tìm kịch bản gây treo hoặc xử lý sai và giải thích.
5. **Dining philosophers.** Cài bài toán 5 triết gia gây deadlock; sửa bằng ba cách: (a) khóa theo thứ tự cố định, (b) `tryLock` có timeout và lùi lại, (c) semaphore giới hạn tối đa 4 người cùng ngồi. So sánh mức độ công bằng và hiệu năng.
6. **Ngắt thread đúng cách.** Viết tác vụ dài có thể hủy bằng `interrupt()`, xử lý `InterruptedException` đúng (khôi phục cờ ngắt, dọn tài nguyên). Viết test chứng minh tác vụ dừng trong thời gian giới hạn, kể cả khi đang `sleep`, đang chờ `BlockingQueue.take()`, và đang chạy vòng lặp tính toán (cần kiểm tra `isInterrupted()`).
7. **Dự đoán.** `Thread t = new Thread(() -> System.out.println("run")); t.run(); t.start(); t.start();` In ra gì và exception nào xảy ra? *Đáp án gợi ý*: `run` in hai lần (lần đầu bởi thread `main` do gọi `run()` trực tiếp, lần hai bởi thread mới), rồi `start()` lần hai ném `IllegalThreadStateException`.
8. **Double-Checked Locking.** Cài singleton DCL có và không có `volatile`. Dùng jcstress cố gắng quan sát object dở dang; nếu không bắt được, giải thích vì sao lỗi hiếm nhưng vẫn là lỗi (phụ thuộc kiến trúc CPU và JIT).
9. **ThreadLocal.** Tái hiện rò rỉ và lẫn dữ liệu giữa request khi dùng `ThreadLocal` trong thread pool mà không `remove`; viết bản sửa bằng `try/finally`.

16.5 java.util.concurrent (Phần 5)

1. **Hành vi ThreadPoolExecutor.** Tạo pool `core=2, max=4, queue=ArrayBlockingQueue(2)`, chính sách mặc định `AbortPolicy`. Nộp 10 task, mỗi task `sleep` 2 giây và in tên thread. Dự đoán trước: task nào chạy ở thread nào, task nào bị từ chối, rồi kiểm chứng. *Đáp án gợi ý*: task 1-2 chạy ở hai thread core; 3-4 vào hàng đợi; 5-6 tạo thêm hai thread (đạt max 4); 7-10 bị từ chối (4 task `RejectedExecutionException`).
2. **Exception bị nuốt.** So sánh `execute` và `submit` khi task ném exception; viết `afterExecute` hoặc wrapper để log mọi lỗi. Kiểm chứng thread worker bị thay thế khi dùng `execute`.

3. **Tự cài với AQS.** (a) Mutex reentrant có `Condition`; (b) `CountDownLatch` bằng chế độ chia sẻ; (c) `Semaphore`. Viết test nhiều thread chứng minh đúng.
4. **ConcurrentHashMap.** Đếm tần suất từ của một file lớn bằng nhiều thread: dùng `HashMap` (quan sát kết quả sai hoặc mất dữ liệu), `Collections.synchronizedMap`, và `ConcurrentHashMap.merge`. So sánh đúng sai và thời gian. Thử viết sai kiểu check-then-act và tìm lỗi.
5. **CompletableFuture.** Giả lập ba dịch vụ chậm (200ms, 500ms, 300ms) và một dịch vụ hay lỗi. Viết hàm gộp kết quả: chạy song song, mỗi dịch vụ có timeout (`orTimeout` / `completeOnTimeout`), có giá trị dự phòng khi lỗi, tổng thời gian xấp xỉ **thời gian dịch vụ chậm nhất** chứ không phải tổng. Dùng executor riêng, kiểm chứng bằng đo thời gian.
6. **Fork/Join.** Cài merge sort song song bằng `RecursiveAction`; thay đổi ngưỡng chia nhỏ, vẽ đồ thị thời gian theo ngưỡng; so sánh với `Arrays.parallelSort`.
7. **Giới hạn đồng thời và tốc độ.** Cài (a) bulkhead bằng `Semaphore` giới hạn N lời gọi đồng thời, (b) rate limiter dạng token bucket. Viết test với 100 thread chứng minh không vượt giới hạn.
8. **Virtual threads.** Chạy 100.000 task mỗi task `sleep` 1 giây trên (a) fixed pool 200 thread, (b) `newVirtualThreadPerTaskExecutor`. Đo tổng thời gian, bộ nhớ, số thread OS. Sau đó thêm khối `synchronized` bao quanh `sleep` trong Java 21 và quan sát **pinning** (dùng `-Djdk.tracePinnedThreads=full`), rồi thay bằng `ReentrantLock`.
9. **Khóa đọc ghi.** Cài cache đọc nhiều ghi ít bằng `synchronized`, `ReentrantReadWriteLock`, `StampedLock` (optimistic read). Benchmark với tỉ lệ đọc 50%, 90%, 99%.

16.6 Generics, Reflection, Annotation, Proxy (Phần 6)

1. **Generics và wildcard.** Cài `static <T extends Comparable<? super T>> T max(Collection<? extends T> c)`; giải thích từng wildcard bằng PECS; tìm một lời gọi chỉ hợp lệ nhờ `? super T`. Tái hiện *heap pollution* qua raw type và varargs generic; dùng `@SafeVarargs` đúng chỗ.
2. **Dự đoán biên dịch.** `List<? extends Number> l = new ArrayList<Integer>(); l.add(1);` và `List<? super Integer> m = new ArrayList<Number>(); m.add(1); Integer i = m.get(0);` Dòng nào lỗi biên dịch và vì sao? *Đáp án gợi ý*: `l.add(1)` lỗi (không biết kiểu cụ thể nên chỉ thêm được null); `m.add(1)` hợp lệ, `Integer i = m.get(0)` lỗi (đọc ra chỉ biết là `Object`).

3. **Mini DI container.** Dùng reflection quét một package, tạo object theo `@Inject` (constructor và field), hỗ trợ singleton và prototype, **phát hiện vòng phụ thuộc** và báo lỗi rõ ràng.
4. **Annotation.** (a) Viết `@NotNull`, `@Range(min,max)` và bộ validator dùng reflection; (b) viết annotation processor lúc compile sinh class `Builder` cho class có `@GenerateBuilder`.
5. **Dynamic proxy.** Viết proxy cho một interface thêm: log + đo thời gian, retry khi gặp exception chỉ định, rồi cache kết quả theo tham số. Chứng minh lỗi **self-invocation**: method A gọi `this.B()` thì B không đi qua proxy.
6. **Serialization.** Tái hiện `InvalidClassException` khi `serialVersionUID` khác; chứng minh `transient` không được lưu; dùng `readResolve` bảo vệ singleton; chặn giải tuần tự hóa kiểu không mong muốn bằng `ObjectInputFilter`.

16.7 Cơ chế bên trong (Phần 8, 9)

1. **JOL.** In layout của `Object`, `int[]`, class có `boolean + long + int`, và class có nhiều field thuộc kiểu khác nhau; chạy với `-XX:-UseCompressedOops` và so sánh. Dự đoán kích thước trước khi chạy, so với kết quả. Giải thích padding.
2. **Mark word và khóa.** In mark word trước, trong và sau `synchronized` (một thread, rồi nhiều thread tranh chấp); nhận diện trạng thái lightweight/ heavyweight. Ghi chú: biased lock đã bị gỡ ở Java 18 nên kết quả phụ thuộc phiên bản.
3. **Memory model.** Dùng jcstress cho các mẫu: message passing (publish bằng cờ), store buffering, và publication của object có field `final`. Với mỗi mẫu, ghi lại các kết quả quan sát và cái nào bị cấm khi thêm `volatile`.
4. **Phân tích GC log.** Sinh log cho ba kịch bản: tải ổn định, tải đột biến, rò rỉ. Viết script (Python hoặc Java) trích **allocation rate**, **promotion rate**, thời gian pause từ log và vẽ biểu đồ. Nhận diện từng kịch bản qua biểu đồ.
5. **AQS nâng cao.** Cài `Semaphore` công bằng và không công bằng, rồi bản `ReentrantReadWriteLock` rút gọn bằng AQS (đọc nhiều, ghi độc quyền). Dùng test nhiều thread để chứng minh không có hai writer cùng lúc và không có reader khi writer đang giữ.
6. **Tự viết ThreadPoolExecutor.** Cài bản tối giản (core, queue, max, keepAlive, reject, shutdown/awaitTermination). Viết bộ test hành vi dựa trên bài 16.5.1 và chạy cả test đó trên bản của JDK để đối chiếu.

7. **Tự viết HashMap.** Cài `MyHashMap<K,V>` (mảng + danh sách, resize khi vượt load factor, tách bucket theo bit). Test với 1.000.000 phép toán ngẫu nhiên, so sánh kết quả với `HashMap` của JDK; thêm đo thời gian khi `hashCode` tệ.
8. **ConcurrentHashMap.** Chứng minh `size()` chỉ là ước lượng tức thời khi có ghi đồng thời (đọc size trong lúc nhiều thread thêm). Cài bộ đếm chia ô (striped counter) kiểu `LongAdder` và so sánh với một biến `AtomicLong` dưới tải cao.
9. **Luyện đọc thread dump.** Tạo bốn kịch bản: deadlock, tranh chấp một khóa, thread pool bị starvation (task chờ task con), và chờ I/O không có timeout. Với mỗi kịch bản chụp thread dump và viết chẩn đoán đúng **ba dòng**: dấu hiệu nhìn thấy, nguyên nhân, cách sửa.

16.8 Dự án tổng hợp

Chọn ít nhất hai dự án. Mỗi dự án cần: mã nguồn có test, báo cáo hiệu năng (JMH hoặc đo tải thực), báo cáo chẩn đoán bằng công cụ JVM (GC log, JFR hoặc thread dump), và một mục "điều tôi đã sai và học được".

Dự án	Nội dung	Tiêu chí hoàn thành
A. HTTP server tối giản	Reactor NIO + thread pool, parser HTTP/1.1, keep-alive, phục vụ file tĩnh bằng `transferTo`, số liệu qua JMX	Chịu 10.000 kết nối đồng thời; so sánh với bản virtual thread; báo cáo p99 và GC
B. Thư viện cache	LRU/LFU đồng thời, TTL, giá trị weak/soft, thống kê, listener khi bị loại, nạp cấu hình qua SPI	Test áp lực đa luồng đúng; JMH so với `LinkedHashMap` có khóa; không rò rỉ bộ nhớ theo MAT
C. Mini framework DI + AOP	Annotation, reflection, dynamic proxy, vòng đời bean, scope, phát hiện vòng phụ thuộc	Có `@Transactional` giả lập bằng proxy; bộ test chứng minh hành vi self-invocation
D. Bộ lập lịch công việc	DAG tác vụ bằng `CompletableFuture`, retry/backoff, hủy, virtual threads, ghi nhật ký tiến trình xuống file (write-ahead log) qua NIO	Mất điện giả lập (kill -9) vẫn khôi phục đúng trạng thái; không có tác vụ chạy hai lần
E. Agent quan sát	`-javaagent` chèn đo thời gian vào method chỉ định (ByteBuddy hoặc ASM), phát sự kiện JFR tùy biến	Overhead đo được dưới ngưỡng bạn đặt; hoạt động với một ứng dụng thật

16.9 Tự đánh giá mức độ làm chủ

Một chủ đề được coi là nắm vững khi bạn **làm được cả ba mức** mà không nhìn tài liệu:

1. **Giải thích**: nói được cơ chế bằng lời, vẽ được sơ đồ (ví dụ: vòng đời một object từ new đến khi bị GC; đường đi của một task trong thread pool).
2. **Tái hiện**: viết được chương trình ngắn gây ra hiện tượng (leak, deadlock, race, OOM, pinning) và chỉ ra bằng công cụ nào thấy được nó.
3. **Khắc phục và đo**: sửa được và chứng minh bằng số liệu rằng bản sửa đúng và hiệu quả.

Nếu mới làm được mức 1, hãy quay lại bài tập tương ứng; kiến thức Java chuyên sâu chỉ thực sự vững khi bạn đã **tự đo, tự làm hỏng và tự sửa** những hiện tượng đó.